Ada Embedded

STM32F4 Discovery Kit edition

Raja Srinivasan

2026-09-19

Table of contents

1. Motivation **1**
- 1.1. Hardware . . . 1
- 1.2. Software Library . . . 1

2. Hello World - Blinker **3**
- 2.1. Projectlet Goals . . . 3
- 2.2. Applications . . . 3
- 2.3. Code walkthrough . . . 3
  - 2.3.1. Basic Blinker . . . 3
  - 2.3.2. External LED Blinker . . . 4
  - 2.3.3. Morse Code Blinker . . . 6
- 2.4. Project Setup . . . 6
  - 2.4.1. Building the project . . . 8
  - 2.4.2. Flashing the binary to the board . . . 8
- 2.5. Development Insights . . . 8

3. Terminal - command language interface **11**
- 3.1. Goals of this project . . . 11
- 3.2. Essential needs . . . 11
  - 3.2.1. Incremental development . . . 11
- 3.3. Import of the support . . . 11
- 3.4. Interaction . . . 12
- 3.5. Serial communication using GPIO pins . . . 14
  - 3.5.1. USART interactions . . . 14
- 3.6. Development Insights . . . 15

4. Logging time **19**
- 4.1. Goals of the project . . . 19
- 4.2. Real time Clock . . . 19
  - 4.2.1. Initialization . . . 20
  - 4.2.2. Adjustment of the clock . . . 20
  - 4.2.3. Consuming the clock . . . 20
- 4.3. Annunciation with the LED . . . 21
- 4.4. Logs transmission . . . 21
  - 4.4.1. Non-blocking IO with Serial Ports . . . 22
  - 4.4.2. Orchestration of IO with interrupts . . . 22
- 4.5. Command Language Interface . . . 24
  - 4.5.1. Design constraints . . . 24
- 4.6. Support library . . . 24
- 4.7. Development Insights . . . 25

5. Data Integrity **27**
- 5.1. CRC support . . . 27
  - 5.1.1. cli session with crc of test vectors . . . 28

5.1.2. Application loop . . . 28
5.2. Implementation . . . 29

6. Library Organization **33**
6.1. Projectlet Goals . . . 33
6.2. Organization of the codebase . . . 33
6.3. Simplified application . . . 34
6.3.1. Example run . . . 35
6.4. Architecture of the library . . . 36
6.4.1. Example: Log service . . . 37

7. Pushbutton **41**
7.1. Projectlet Goals . . . 41
7.2. Dual applications . . . 41

8. Analog world **43**
8.1. Projectlet Goals . . . 43
8.1.1. Setup the conversions . . . 43
8.1.2. Interpreting the ADC output . . . 44
8.1.3. Measurement Accuracy . . . 44
8.1.4. Experimental Setup and Results . . . 45
8.2. Development Insights . . . 50

9. Interrupts and Timers **51**
9.1. Projectlet Goals . . . 51
9.1.1. Basic Approach . . . 51
9.2. Implementation . . . 52
9.2.1. Toolkit Support . . . 52
9.2.2. Interfacing the Interrupts and application . . . 52
9.2.3. Pulse width modulation and fading the LED . . . 53
9.2.4. Log generation . . . 54
9.3. Results . . . 55
9.4. Development Insights . . . 56
9.4.1. Inter task messaging . . . 56

10. Data Logging **59**
10.1. Projectlet Goals . . . 59
10.1.1. Protocol for binary data communications . . . 59
10.1.2. Data Integrity, Framing and Synchronization . . . 59
10.2. Implementation . . . 60
10.2.1. Payload Handling . . . 60
10.2.2. Transport . . . 61
10.3. Diagnostic Toolset . . . 63
10.4. Development Insights . . . 64

11. Digital Communications - I2C **67**
11.1. Projectlet Goals . . . 67
11.1.1. The roadmap . . . 67
11.2. Sensor Network . . . 69
11.2.1. Temperature Sensor stts22h . . . 69
11.2.2. Alphanumeric Display ht16k33 . . . 71

© Raja Srinivasan. All rights reserved.

Appendices **77**

A. Development Environment **77**
A.1. Application code repository 77
A.1.1. Book source 77
A.1.2. Target microcontrollers 77
A.1.3. Usage 78
A.1.4. Building the applications 78
A.1.5. Code deployment or flashing the target 78
A.1.6. Code example extracts 78
A.1.7. Building the documentation 78
A.1.8. Data Analysis 79

© Raja Srinivasan. All rights reserved.

1. Motivation

This project is a follow on to the earlier adaskills book. The book was taking baby steps in applying Ada to complex problems. By no means was it complete leaving many interesting features unexplored. Efforts are underway to revise the book and make it more comprehensive.

A particular gap in coverage was **embedded systems** loosely interpreted here as microcontroller based systems - potentially bare metal or perhaps with other **RTOS**es. This project is an attempt to fill that gap.

By nature, embedded systems are resource constrained and time sensitive. They are ***real time*** systems sometimes with hard real time requirements and constraints. Typical applications tend to be safety critical and require high reliability over essentially forever. Ada is a natural fit for such applications and has been used in many such applications over the years.

My own interest in this area started in the mid 80s when I was fortunate to be part of the team involved in designing a complex and large steel mill control system. The previous design built with **Fortran** was ripe for revision and we were given the opportunity to design a new system from scratch. The team chose Ada and an **RTOS** as the target platform collaborating with a supervisory layer that benefited from a conventional general purpose operating system. The system grew to be the first major non defense application comprising more than half a million lines of code. It was deemed a success and continued to control steel mills for many years.

1.1. Hardware

The nostalgia of those days is rekindled these days with the availability of powerful microcontrollers and the ability to run Ada on them. The prohibitive cost of the development platform is no longer a barrier for hobbyists and learners. This book is an attempt to explore the use of Ada in practical embedded applications. While there are many potential microcontroller platforms, the STM32F4 Discovery board is a good choice for learning and experimentation. It is widely available, affordable, and has a rich set of features that make it suitable for a variety of applications. My own toolchest contained an STM32F4 discovery board from several years ago and Ada was supported on it.

1.2. Software Library

The software library available Ada Drivers Library is rich and supports many microcontrollers besides the STM32F4. The intent is to build another layer on top of the ADL to enable us to develop full fledged applications with ease. Based on the experience of many years of developing medical devices, there are always a core set of features that teams spend a considerable effort building and maintaining. These include a command line interface, logging, reliable data communications and so on. The intent is to build a reusable library that provides these features and more on top of the ADL.

2. Hello World - Blinker

```
This chapter is based on the projectlet:

- 01_button
```

2.1. Projectlet Goals

- Setup the development environment targeting the chosen hardware namely the STM32F4 Discovery board. The ultimate resource for this is of course the **AdaCore** documentation and we will not address these here. For convenience there are **Docker** containers that are available from a companion repository devops which has been used extensively in developing this book. The appendix provides an overview of the approach.
- Experiment with the basic toolset for building, flashing the application and testing.

2.2. Applications

The equivalent of the well known "Hello World" program in the embedded world is the "blinker" - essentially a demonstration of how to interact with the **GPIO pins** - specifically to blink an LED. Leaving the hardware details aside, the software simply needs to set the output of the GPIO pin to High and Low to turn the LED on and off. Blinking then is achieved by adding a delay between the High and Low states.

In this project, we apply the Ada Driver Library adl to this problem. The library itself supports many microcontroller families and boards, and we focus in this project on the **STM32F4 Discovery** board. The git repository is setup as a submodule of this repository for easy access.

Three variations on this theme are provided for illustration:

- **blinker** - the basic blinker application dealing with the LEDs on the board itself.
- **blinker_ext** - essentiall the same as above except the LED itself is external to the board
- **blinker_morse** - adds a different blinking pattern to the mix - in this case a *Morse code* pattern. Such an application is used in the real world to signal different challenges faced by the application.

2.3. Code walkthrough

2.3.1. Basic Blinker

The environmetal setup for this simple application - including the necessary parts of the **adl** are specified:

Listing 2.1 Basic environment

```
File : blinker.adb

0002 | --   The packages referenced here are part of the Ada Driver Library.
0003 | --   Only the STM32 relevant packages will be pulled in by the build system
0004 | --   though the library itself is quite large and
0005 | --   supports quite a few microcontroller families.
0006 | with STM32.Board;
0007 | with STM32.GPIO;
0008 | with STM32.Device;
```

In particular the package **STM32.Board** indicates LEDs available on the board - ie the 4 LEDS - blue, green, red and orange. In this example, the LEDs are lit in turns forming an anticlockwise pattern as we view the board:

Listing 2.2 Primary function loop

```
File : blinker.adb

0019 |    STM32.Board.Initialize_LEDs;                    -- Onboard LEDs are initialized.
0020 |    stm32.board.All_LEDs_Off;
0021 |    loop
0022 |       -- Toogle the state of one LED and after a delay, toggle it again
0023 |       --  and then onto the next LED.
0024 |       stm32.gpio.Toggle (stm32.Board.Green_LED);
0025 |       delay 0.5;
0026 |       stm32.gpio.Toggle (stm32.Board.Green_LED);
0027 |       stm32.gpio.Toggle (stm32.Board.Blue_LED);
0028 |       delay 0.5;
0029 |       stm32.gpio.Toggle (stm32.Board.Blue_LED);
0030 |       stm32.gpio.Toggle (stm32.Board.Red_LED);
0031 |       delay 0.5;
0032 |       stm32.gpio.Toggle (stm32.Board.Red_LED);
0033 |       stm32.gpio.Toggle (stm32.Board.Orange_LED);
0034 |       delay 0.5;
0035 |       stm32.gpio.Toggle (stm32.Board.Orange_LED);
0036 |    end loop;
```

The library **STM32.GPIO** supports manipulating the *GPIO pins*.

2.3.2. External LED Blinker

Tbe board provides numerous GPIO pins and many (if not most) of them can be used to connect other devices. In this example we use an external LED and attempt to blink the LED. Of course in this case, we need to tell the system the specific pins where the LED is connected and initialize those explicitly:

© Raja Srinivasan. All rights reserved.

Listing 2.3 External LED specification

```
File : status.ads

0006 | package status is
0007 |    LED : STM32.Board.User_LED renames STM32.Device.PC1 ;
0008 |    procedure Initialize( dev : in out STM32.Board.User_LED);
0009 |    procedure Toggle( dev : in out STM32.Board.User_LED) ;
0010 | end status ;
```

In this and related projects, the moniker **status** is used to refer to the external LED. The initialization similar to the OnBoard LEDs :

Listing 2.4 Initialization of the external LED

```
File : status.adb

0004 |    procedure Initialize( dev : in out STM32.Board.User_LED) is
0005 |    begin
0006 |       Enable_Clock (dev) ;
0007 |       stm32.GPIO.Configure_IO
0008 |         (dev,
0009 |          (stm32.gpio.Mode_Out,
0010 |           Resistors   => stm32.gpio.Floating,
0011 |           Output_Type => stm32.gpio.Push_Pull,
0012 |           Speed       => stm32.gpio.Speed_100MHz));
0013 |       stm32.GPIO.Clear(dev) ;
0014 |    end Initialize;
```

Once initialized, the external LED is dealt with in the same way as any other LED.

Listing 2.5 Primary loop with external LED

```
File : blinker_ext.adb

0012 |    status.Initialize( status.LED );
0013 |    loop
0014 |       delay 0.5;
0015 |       status.Toggle ( status.LED );
0016 |    end loop;
```

During development of course, a **proto board** is a flexible testing solution:

© Raja Srinivasan. All rights reserved.

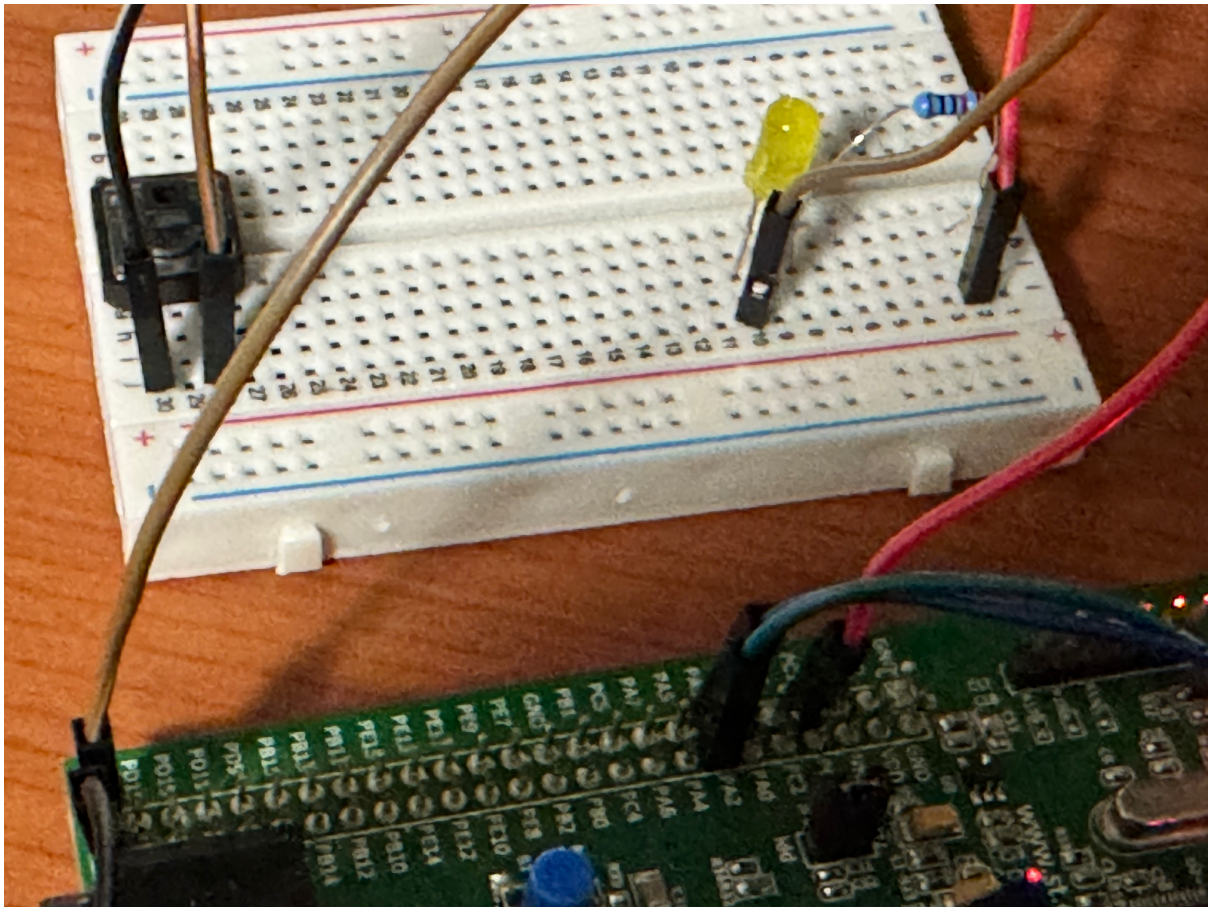


Figure 2.1.: External LED

2.3.3. Morse Code Blinker

Introducing some variations, given a character, its representation in morse code is signaled in this project. The key parameter for this application is the duration of **On** and **Off** patterns:

A simple application signals just one character :

In real life examples, particularly when there is a lack of convenient output, this can be used to signal an internal condition.

2.4. Project Setup

Standard environment created by **alire** is modified a little for this project. The pattern is used in the initial set of projects in this group till a more flexible approach is evolved.

File : blinker.gpr

```
0002 | with "../../adl/boards/stm32f407_discovery/stm32f407_discovery_full.gpr";
0003 | with "../common.gpr" ;
0004 | with "config/blinker_config.gpr";
0005 |
0006 | project Blinker is
```

© Raja Srinivasan. All rights reserved.

Listing 2.6 Morse code using an LED

File : morsecode.adb

```
0143 |    procedure Generate (C : Character; led : in out STM32.Board.User_LED)
0144 |    is
0145 |       use STM32.GPIO ;
0146 |       -- The letter translates into a sequence of symbols in this code.
0147 |       code : Letter_Representation := Translate (C);
0148 |    begin
0149 |       for a in code'Range loop
0150 |          case code (a) is
0151 |          -- Each symbol dot and dash are distinguished by the duration
0152 |          --  when an LED is set and after each symbol is a delay
0153 |          --  when the LED is cleared.
0154 |             when Dot =>
0155 |                Set (led) ;
0156 |                delay Span (Dot);
0157 |                Clear( led );
0158 |                delay Span (minigap);
0159 |             when Dash =>
0160 |                Set (led);
0161 |                delay Span (Dash);
0162 |                Clear(led);
0163 |                delay Span (minigap);
0164 |             when others =>
0165 |                null;
0166 |          end case;
0167 |       end loop;
0168 |       -- Once all the symbols of a letter are generated,
0169 |       --  there is an inter letter gap before the next letter is generated.
0170 |       delay Span (gap);
0171 |    end Generate;
```

```
0007 |
0008 |    for Target use "arm-eabi" ;
0009 |    for Runtime ("Ada") use "embedded-stm32f4" ;
0010 |
0011 |    for Source_Dirs use ("src/", "config/");
0012 |    for Object_Dir use "obj/" & Blinker_Config.Build_Profile;
0013 |    for Create_Missing_Dirs use "True";
0014 |    for Exec_Dir use "bin";
0015 |    for Main use ("blinker.adb" , "blinker_ext.adb" , "blinker_morse.adb") ;
0016 |    package Compiler is
0017 |       for Default_Switches ("Ada") use common.Compiler_Switches ;
0018 |    end Compiler;
```

Only a subset of the entire ADL is referenced here - the parts relevant to **STM32F4 Discovery** boards.

© Raja Srinivasan. All rights reserved.

Listing 2.7 Potential application

```
File : blinker_morse.adb

0007 | procedure Blinker_morse is
0008 |    message : constant String := "A" ;
0009 | begin
0010 |    status.Initialize( status.LED );
0011 |    for c of message loop
0012 |       Morsecode.Generate (c, status.LED);
0013 |    end loop;
0014 | end Blinker_morse ;
```

In addition, all the projects in this group will use standard compiler settings which are specified in the **common.gpr** file included here.

2.4.1. Building the project

With the setup outlined above, building the project is straightforward:

```
alr build
```

which produces the **elf** images for all the 3 applications.

2.4.2. Flashing the binary to the board

In this group of projects, **openocd** is the tool used for flashing the binary. In the case of STM32 boards, **st-link** is also a possibility.

```
openocd -f interface/stlink.cfg \
        -f target/stm32f4x.cfg \
        -c 'init; reset halt; program bin/blinker verify reset exit'
```

In the example above, the **elf** file generated by the linker is the input for openocd. The elf file contains all the information such as the load address that is needed to flash. In addition the debug symbols are also embedded in the elf file and can be used for debugging.

2.5. Development Insights

This projectlet feels very similar to conventional software development. The tools used in the process are rather close. Even the applications have the same feel. **alr** is the central driver, creating our project infrastructure, driving the builds and so on. The only **magic** is the binding to the physical LED. The component specification will of course differ based on the board. The fact that there are 4 LEDs of colors Red, Blue, Green and Orange each with their **id** are very significant. In our toolchain the description of the board **STM32F4 Discovery**, its components are described in:

© Raja Srinivasan. All rights reserved.

```
$ADL/arch/ARM/STM32/devices/stm32f40x/stm32-device.ads
$ADL/boards/stm32_common/stm32f407disco/stm32-board.ads
ADL -> the directory location of the Ada Driver Library
```

Clearly the low level **device** specification and the slightly more specific **board** specific descriptions are both the source of truth for our **bare metal** projects in this book. As we explore more and more of the low level devices/interfaces such as **I2C**, **I2S**, **ADC** these will be our primary references.

Of course, the ultimate authoritative tool is the STM32MxCube for the descriptions and application notes. In addition the STM32Cube IDE is useful for exploration and experimentation. We will be visiting these tools in later more interesting projects.

© Raja Srinivasan. All rights reserved.

3. Terminal - command language interface

```
This chapter is based on the projectlet:
02_cli
```

3.1. Goals of this project

In this projectlet, consideration of debugging of practical real applications is the focus. All embedded applications need a way to communicate statuses, ability to parametrize operations and so on. During development it is critical to make it easy and convenient to add diagnostic features. A command language interface support is developed.

3.2. Essential needs

A **cli** support needs to be somewhat similar to a **shell** in a richer environment like **Linux**. User enters a command and the system responds with an answer. In the embedded world, the hardware support utilized is a pair of **GPIO** pins - one to transmit and another to receive using a simple serial interface protocol.

The **adl** has provided some examples of implementing such a serial interaction. For simplicity we adopt a ***blocking*** protocol for this communication. The choice of using interrupts to avoid blocking will be illustrated in a later project.

3.2.1. Incremental development

This project and other subsequent projects will also build upon the previous project the **blinker** to ensure the architecture being evolved is flexible.

3.3. Import of the support

The support for the basic Serial Communication is not in theory part of the **adl** but more of the **examples** component of the adl. Hence we copy the relevant code into a directory called **imports** thus affording us the opportunity to adapt the code to our needs. This pattern is used in other projects as a practise.

Listing 3.1 Main Loop of interactions

```
File : cli.adb

0014 |
0015 | with terminal;                          -- fundamental operations are abstracted in
0016 |
0017 | procedure cli is
0018 | begin
0019 |    loop
0020 |       declare
0021 |          Req : String := terminal.Receive;
0022 |       begin
0023 |          cmd.Execute( Req );
0024 |       end ;
0025 |    end loop ;
```

3.4. Interaction

Once the basic support for serial communication is available, the classic **REPL** can be implemented:

In its essence the system needs a way to send output to the user and another to accept a command. In this implemenation, it is fully simplex - no parallel transmissions - supported using the interface spec :

Listing 3.2 Command Line I/O support

```
File : terminal.ads

0005 |    function  Receive return String ;
0006 |    procedure Send (This : String);
```

The user entered line is analyzed and dispatched to an implementing function as follows:

An example command **random** may be implemented as:

As shown above, the actual random number generation utilizes the **hardware** random number generator provided by the STM32 as indicated:

Slightly more involved might be multiword commands eg. :

To summarize, a **REPL** module can be added to the application and will be extremely useful as a diagnostic tool during development. It is not anticipated that such support will be released to production - except with extreme precautions.

This implementation suffers from the limitation that it is “simplex” in that the primary logic is single threaded and when the system is awaiting user input, no other operations can go on. In later improvements this restriction will be lifted.

Following is an example of an interactive session with the **cli**:

© Raja Srinivasan. All rights reserved.

Listing 3.3 Dispatch based on the command

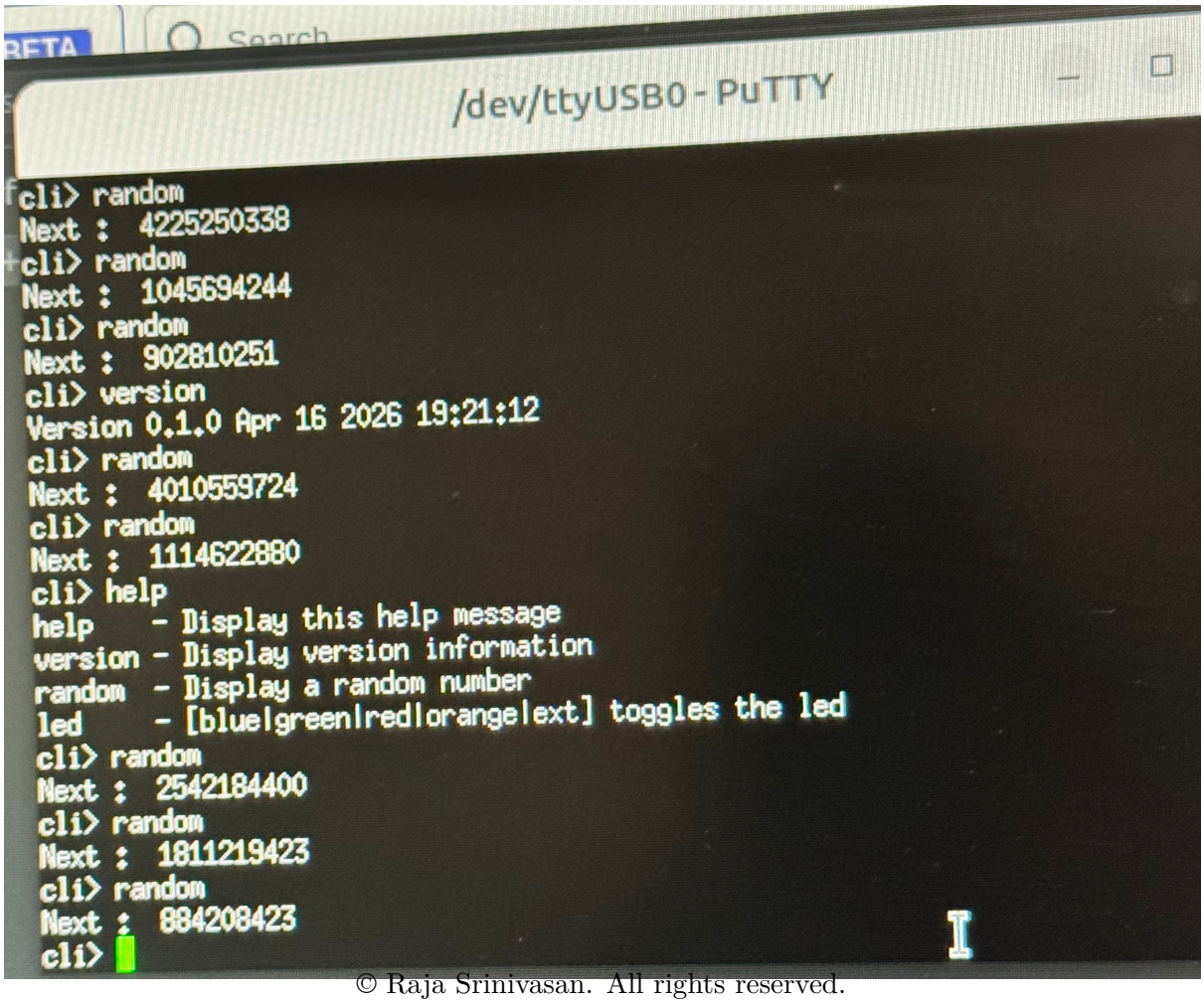
File : cmd.adb

```
0073 |       if cmdline( strfrom .. strto ) = "help" then
0074 |          Help ;
0075 |          return ;
0076 |       elsif cmdline( strfrom .. strto ) = "version" then
0077 |          Version ;
0078 |          return ;
0079 |       elsif cmdline( strfrom .. strto ) = "random" then
0080 |          Random ;
0081 |          return ;
```

Listing 3.4 Generate the next random number

File : cmd.adb

```
0115 |    procedure Random is
0116 |    begin
0117 |       New_Line ;
0118 |       terminal.Send("Next : " ) ;
0119 |       terminal.Send( support.rng.Random'Image ) ;
0120 |       New_Line ;
0121 |    end Random ;
```



© Raja Srinivasan. All rights reserved.

Figure 3.1.: Putty session

Listing 3.5 Use the hardware RNG from STM32

```
File : support-rng.adb

0004 |    function Random return Interfaces.Unsigned_32 is
0005 |    begin
0006 |       return Interfaces.Unsigned_32(STM32.RNG.Interrupts.Random) ;
0007 |    end Random ;
```

Listing 3.6 Multiword commands and dispatch

```
File : cmd.adb

0128 |       Split ( cmdline , work , strfrom , strto ) ;
0129 |       if strfrom > cmdline'last
0130 |       then
0131 |          New_Line ;
0132 |          terminal.Send("which LED?");
0133 |          New_Line ;
0134 |          return ;
0135 |       end if ;
0136 |
0137 |       if cmdline( strfrom .. strto ) = "blue" then
0138 |          stm32.GPIO.Toggle( BLUE_LED ) ;
0139 |       elsif cmdline( strfrom .. strto ) = "red" then
0140 |          stm32.GPIO.Toggle( RED_LED ) ;
0141 |       elsif cmdline( strfrom .. strto ) = "green" then
0142 |          stm32.GPIO.Toggle( GREEN_LED ) ;
0143 |       elsif cmdline( strfrom .. strto ) = "orange" then
0144 |          stm32.GPIO.Toggle( orange_LED ) ;
0145 |       elsif cmdline( strfrom .. strto ) = "ext" then
0146 |          stm32.GPIO.Toggle( support.led.status_led ) ;
```

3.5. Serial communication using GPIO pins

The serial communication channel is specified with a number of parameters. Thereafter they can be configured as needed:

The specifications are very similar for most microcontrollers though some may require additional specifications or may be simpler. This arises out of the convenience that most GPIO pins can be applied for different functions depending on the peripheral device attached. A GPIO pin may be TX for one setup but independently may serve as a member of an I2C bus in another. In this project the specifications are:

3.5.1. USART interactions

Since the communication is designed to be **Simplex** the state of the peripheral should be monitored to know when the transmitter or the receiver is **idle** so the next step can be performed. The monitoring of the peripheral is achieved thus:

© Raja Srinivasan. All rights reserved.

Listing 3.7 Specification for the interface

```
File : serial_io.ads

0040 |    type Peripheral_Descriptor is record
0041 |       Transceiver    : not null access USART;
0042 |       Transceiver_AF : GPIO_Alternate_Function;
0043 |       Tx_Pin         : GPIO_Point;
0044 |       Rx_Pin         : GPIO_Point;
0045 |    end record;
0046 |
0047 |    -- Configuration for the Serial port
0048 |    -- Typical:
0049 |    --   115200 baud, 8 data bits,  1 stop bit, no parity
0050 |    --   no flow control
0051 |     procedure Configure
0052 |      (Device    : access USART;
0053 |       Baud_Rate : Baud_Rates;
0054 |       Parity    : Parities     := No_Parity;
0055 |       Data_Bits : Word_Lengths := Word_Length_8;
0056 |       End_Bits  : Stop_Bits    := Stopbits_1;
0057 |       Control   : Flow_Control := No_Flow_Control);
0058 |
```

The state of the peripheral **USART** is utilized in achieving transmission and reception of data:

3.6. Development Insights

Using a ***blocking*** discipline for serial communications implies that at any instant, the app is:

- waiting for transmission of the prompt to complete
- waiting for the reception of the user input to complete
- executing the command

Thus the LEDs are not able to blink independently. LEDs can be toggled as part of the command execution.

Real applications will have to look at ***non blocking*** approaches to enable parallel operations.

In this projectlet the random number generator onboard has been utilized for illustrative purposes. The performance of this should be evaluated for suitability in any application.

© Raja Srinivasan. All rights reserved.

Listing 3.8 GPIO pins for interaction

```
File : peripherals_blocking.ads

0041 |    --  the USART selection is arbitrary but the AF number and the pins must
0042 |    --  be those required by that USART
0043 |    Peripheral : constant Serial_IO.Peripheral_Descriptor :=
0044 |                    (Transceiver    => USART_1'Access,
0045 |                     Transceiver_AF => GPIO_AF_USART1_7,
0046 |                     Tx_Pin         => PB6,
0047 |                     Rx_Pin         => PB7);
0048 |
0049 |    COM : Blocking.Serial_Port (Peripheral.Transceiver);
```

Listing 3.9 Monitoring the USART for availability

```
File : serial_io-blocking.adb

0068 |    procedure Await_Send_Ready (This : access USART) is
0069 |    begin
0070 |       loop
0071 |          exit when This.Tx_Ready;
0072 |       end loop;
0073 |    end Await_Send_Ready;
0074 |
0075 |    procedure Await_Data_Available (This : access USART) is
0076 |    begin
0077 |       loop
0078 |          exit when This.Rx_Ready;
0079 |       end loop;
0080 |    end Await_Data_Available;
```

© Raja Srinivasan. All rights reserved.

Listing 3.10 Low level transmit and receive operations

File : serial_io-blocking.adb

```
0037 |    procedure Send (This : in out Serial_Port;
0038 |                    Msg : not null access Message) is
0039 |    begin
0040 |       for Next in 1 .. Msg.Length loop
0041 |          Await_Send_Ready (This.Device);
0042 |          This.Device.Transmit
0043 |            (Character'Pos (Msg.Content_At (Next)));
0044 |       end loop;
0045 |    end Send;
0046 |
0047 |    procedure Receive (This : in out Serial_Port;
0048 |                       Msg : not null access Message) is
0049 |       Received_Char : Character;
0050 |       Raw           : HAL.UInt9;
0051 |    begin
0052 |       Msg.Clear;
0053 |       Receiving : for K in 1 .. Msg.Physical_Size loop
0054 |          Await_Data_Available (This.Device);
0055 |          This.Device.Receive (Raw);
0056 |          Received_Char := Character'Val (Raw);
0057 |          Await_Send_Ready (This.Device);
0058 |          This.Device.Transmit
0059 |            (Character'Pos (Received_Char));
0060 |          exit Receiving when Received_Char = Msg.Terminator;
0061 |          Msg.Append (Received_Char);
0062 |       end loop Receiving;
0063 |    end Receive;
```

© Raja Srinivasan. All rights reserved.

4. Logging time

```
This chapter is based on the projectlet
03_logger
```

4.1. Goals of the project

As the design of an embedded application evolves and we begin to tackle diagnostic needs with tools such as a cli as the first step. The next most critical need is annunciating states and measurements. Proper diagnostics will need a timestamp, events and data in a predictable order from any of the applications implemented in the system. Enumerating the requirements then:

- A clock function is needed that maintains a **real time** clock ie matching the external clock time. The controllers provide **tick**s which is an indication of the progression but is of questionable value in system diagnostics.

- A logging channel that is used by the application. Different functions within the application may be operating independently eg as threads (**task**s in **Ada** terminology). Thus the logging facility must be inherently ***thread safe***.

- The functions implemented in earlier projects - the blinker and the cli were not **multi-threaded** and thus only those functions could operate at any time. The terminal (in the cli project) has to wait for user input but cannot hold up a motor control thread (**task**) for instance. Similarly blinking an LED should not hold up other application functions.

In this project, some of the above requirements are addressed - on an experimental basis while the main logging function itself does not benefit from the parallelizing effort. The logging function is simplistic in this project - just logging the clock periodically in its main thread/task.

4.2. Real time Clock

The STM32F4 discovery board incorporates an **RTC** (real time clock) that maintains a **calendar**. This is not extremely accurate and most real applications will probably utilizing a higher accuracy chip and connect an accurate high frequency clock signal in order to maintain the clock. However the onboard RTC (using LSI clocking) will be sufficient for experimentation.

Listing 4.1 Interface to the RTC calendar and clock

```
File : support-rtclsi.ads

0006 |    type day_name is (Monday, Tuesday, Wednesday, Thursday, Friday,
0007 |                       Saturday, Sunday );
0008 |    pragma Ordered( day_name );
0009 |
0010 |    procedure Set_Date (Year, Month, Day, Weekday : UInt8) ;
0011 |    procedure Set_Time (Hour, Minute, Second : UInt8) ;
0012 |    procedure Get_Time (Hour, Minute, Second : out UInt8) ;
0013 |    procedure Get_Date (Year, Month, Day, Weekday : out UInt8);
0014 |    function Get_Day( weekday : UInt8 ) return String ;
0015 |    function Get_Date return String ;
0016 |    function Get_Time return String ;
0017 |    function TodayName return String ;
```

4.2.1. Initialization

In embedded applications that rely on a clock, it is essential to design in a nonvolatile storage of the clock ie across a power cycle, the clock shall be maintained instead of having to be reinitialized. However in the STM32F4 discovery board, the clock needs initialization with a date, time and day of the week. Once initialized the clock is maintained and can be queried as needed.

with that the clock is initialized:

Listing 4.2 Simple calendar using onboard RTC

```
File : logger.adb

0020 |    support.rtclsi.Set_Date (Year => 26, Month => 03, Day => 19,
0021 |                             Weekday => 3) ;
0022 |    support.rtclsi.Set_Time (Hour => 10, Minute => 50 , Second => 0) ;
```

4.2.2. Adjustment of the clock

It is to be noted that the rtc's accuracy spec will lead to a **drift** in the clock over a period of time. If a **system** has multiple clocks, each drifting at a different rate then a system mechanism will have to be implemented to ensure period synchronization. In this project we provide a cli command to set the date and time which will suffice for experimentation.

By necessity, the initialization above uses a hardcoded date and time and in experimentation, one of the first user actions will be to use the cli to set the clock.

4.2.3. Consuming the clock

With the above initializations, the clock can be polled periodically and **logged**.

© Raja Srinivasan. All rights reserved.

Listing 4.3 Simple and boring logging

```
File : logger.adb

0034 |    loop
0035 |       support.logs.Put_Line( "Current time is: " &
0036 |                              support.rtclsi.Get_Time ) ;
0037 |       delay 2.0 ;
0038 |    end loop ;
```

4.3. Annunciation with the LED

In the previous projects, the LED blinking would end up blocking the entire controller. In this project, each LED is operated by an independent **task** with its own parameters. In a simplistic implementation, only the period of **On** and **Off** are used for illustration. The key learning point of this exercise is to make the operation independent - a **task** and provide a mechanism - messaging - to communicate with the task.

Listing 4.4 Independent parallel operation of the LEDs

```
File : logger.adb

0026 |    support.led.LED_Parameters.Set_Pattern( green_led_id , 'G' ,  0.5 ) ;
0027 |    support.led.LED_Parameters.Set_Pattern( red_led_id , 'R' ,  1.5 ) ;
0028 |    support.led.LED_Parameters.Set_Pattern( blue_led_id , 'B' ,  2.5 ) ;
0029 |    support.led.LED_Parameters.Set_Pattern( orange_led_id , 'O' ,  3.5 ) ;
0030 |    support.led.LED_Parameters.Set_Pattern( status_led_id , 'S' ,  4.5 ) ;
```

The above segment parameterizes the led tasks referred to by their **id**s. Each line targets a specific LED and the appropriate task can retrieve the appropriate parameter in its operation.

In conventional platforms, the above **message**ing will be achieved by a combination of **entry**, **accept** and other such features. In the **runtime** that is used in these projects namely **embedded-stm32f4**, they are not supported and an alternative is needed. We use a **protected** intermediary object for this purpose.

The project file provides the following specifications :

```
File : logger.gpr

0008 |    for Target use "arm-eabi" ;
0009 |    for Runtime ("Ada") use "embedded-stm32f4" ;
```

4.4. Logs transmission

For transmission of the logs, we can utilize one of the **USART** or **UART** peripherals with the **GPIO** pins. Beyond the low level interface, the synchronous discipline adopted for the earlier project **cli** is no longer suitable. The need is to be non blocking so multiple operations can be in

© Raja Srinivasan. All rights reserved.

progress simultariously. At this level, the logging operation and cli operation are independent consumers of the USART/UART peripherals.

Agail the adl provides examples of such non blocking serial io operations which is replicated in this project, affording us the option to adopt as necessary.

4.4.1. Non-blocking IO with Serial Ports

From a user's perspective, a channel is defined as a combination of USART and the pins for TX and RX operation. The alternative to polling for operation completion is to connect to an appropriate interrupt - thus enabling other operatings to continue. The specifications for such an interface are:

Listing 4.5 Non-Blocking Serial I/O interface

```
File : serial_io-nonblocking.ads

0050 | package Serial_IO.Nonblocking is
0051 |    pragma Elaborate_Body;
0052 |
0053 |    type Serial_Port
0054 |      (Device       : not null access USART;
0055 |       IRQ          : Interrupt_ID;
0056 |       IRQ_Priority : Interrupt_Priority)
0057 |    is limited private;
0058 |
0059 |    procedure Send
0060 |      (This : in out Serial_Port;
0061 |       Msg  : not null access Message);
0062 |    --  Start sending the content of Msg.all, returning potentially
0063 |    --  prior to the completion of the message transmission
0064 |
0065 |    procedure Receive
0066 |      (This : in out Serial_Port;
0067 |       Msg  : not null access Message)
0068 |    with
0069 |       Post => Msg.Length <= Msg.Physical_Size and
0070 |               (if Msg.Length > 0 then
0071 |                   Msg.Content_At (Msg.Length) /= Msg.Terminator);
0072 |    --  Start receiving Msg.all content, ending when the specified
0073 |    --  Msg.Terminator character is received or the physical capacity
0074 |    --  of Msg.all is reached. The terminator character is not stored.
```

Though the specificiation details above are public, behind the scenes, the access to the port has to be exclusive implemented using the **protected** construct:

4.4.2. Orchestration of IO with interrupts

Since blocking on the IO operations is not desired, **interrupt**s are leveraged to achieve a non blocking transmission. The application connects to the appropriate Interrupt and the processor

© Raja Srinivasan. All rights reserved.

Listing 4.6 Exclusive access to each serial port

File : serial_io-nonblocking.ads

```
0079 |    protected type Serial_Port
0080 |      (Device       : not null access USART;
0081 |       IRQ          : Interrupt_ID;
0082 |       IRQ_Priority : Interrupt_Priority)
0083 |    with
0084 |       Interrupt_Priority => IRQ_Priority
0085 |    is
0086 |
0087 |       procedure Start_Sending (Msg : not null access Message);
0088 |
0089 |       procedure Start_Receiving (Msg : not null access Message);
0090 |
0091 |    private
0092 |
0093 |       Next_Out     : Positive;
0094 |       Outgoing_Msg : access Message;
0095 |       Incoming_Msg : access Message;
0096 |
0097 |       procedure Handle_Transmission with Inline;
0098 |       procedure Handle_Reception    with Inline;
0099 |       procedure Detect_Errors (Is_Xmit_IRQ : Boolean) with Inline;
0100 |
0101 |       procedure ISR with Attach_Handler => IRQ;
0102 |
0103 |    end Serial_Port;
```

delivers the interrupt to the specified **interrupt service routine** (ISR).

Once setup transmission of a message proceeds:

In the above, setting up ***Next_Out*** initiates the transmissions - one character at a time starting with the first. Every ***Transmission_Complete*** interrupt should initiate the transmission of the next character and so on:

The operations continue and when all the required octet's are transmitted, the application is informed by means of the ***Message*** object.

Message transmission proceeds in a similar fashion. In this application, a line terminator is designated and when the terminator is received, the message reception is considered complete. In this application, the logger channel does not handle inputs though an **RX** pin is designated. In the case of the **cli** which leverages the same features will utilize both - transmission and reception by means of the non blocking discipline.

The pattern above is pretty common at the low level of most embedded applications.

© Raja Srinivasan. All rights reserved.

Listing 4.7 Non Blocking Interrupt Handling

File : serial_io-nonblocking.adb

```
0110 |       procedure ISR is
0111 |       begin
0112 |          --  check for data arrival
0113 |          if Device.Status (Read_Data_Register_Not_Empty) and then
0114 |            Device.Interrupt_Enabled (Received_Data_Not_Empty)
0115 |          then
0116 |             Detect_Errors (Is_Xmit_IRQ => False);
0117 |             Handle_Reception;
0118 |             Device.Clear_Status (Read_Data_Register_Not_Empty);
0119 |          end if;
0120 |
0121 |          --  check for transmission ready
0122 |          if Device.Status (Transmission_Complete_Indicated) and then
0123 |            Device.Interrupt_Enabled (Transmission_Complete)
0124 |          then
0125 |             Detect_Errors (Is_Xmit_IRQ => True);
0126 |             Handle_Transmission;
0127 |             Device.Clear_Status (Transmission_Complete_Indicated);
0128 |          end if;
0129 |       end ISR;
```

4.5. Command Language Interface

With the non blocking serial IO capabilities implemented, the main loop of the **cli** project is easily converted into a **task** functioning in parallel with the application - which is simple logging in this application :

4.5.1. Design constraints

The design implemented and outlined above has imposed certain constraints such as a line terminator for receptions and handling of overflow conditions when the line terminator is not received. Later projects will illustrate how to handle arbitrary length messages with no line terminator designated.

Integrity requirements may also lead to adding a **CRC** based error checking to account for the inevitable glitches.

4.6. Support library

As we are expanding the footprint of the applications, we can see a library evolving. A family of packages under the moniker **support** is part of this project:

```
support.ads
support-cli.ads
```

© Raja Srinivasan. All rights reserved.

Listing 4.8 Non blocking transmission

File : serial_io-nonblocking.adb

```
0136 |       procedure Start_Sending (Msg : not null access Message) is
0137 |       begin
0138 |          Outgoing_Msg := Msg;
0139 |          Next_Out := 1;
0140 |
0141 |          Device.Enable_Interrupts (Parity_Error);
0142 |          Device.Enable_Interrupts (Error);
0143 |          Device.Enable_Interrupts (Transmission_Complete);
0144 |       end Start_Sending;
```

Listing 4.9 Non Blocking - transmission

File : serial_io-nonblocking.adb

```
0064 |       procedure Handle_Transmission is
0065 |       begin
0066 |          --  if Word_Lenth = 9 then
0067 |          --    -- handle the extra byte required for the 9th bit
0068 |          --  else  -- 8 data bits so no extra byte involved
0069 |          Device.Transmit(Character'Pos
0070 |                              (Outgoing_Msg.Content_At(Next_Out)));
0071 |          Next_Out := Next_Out + 1;
0072 |          --  end if;
0073 |          if Next_Out > Outgoing_Msg.Length then
0074 |             Device.Disable_Interrupts (Source => Transmission_Complete);
0075 |             Outgoing_Msg.Signal_Transmission_Complete;
0076 |             Outgoing_Msg := null;
0077 |          end if;
0078 |       end Handle_Transmission;
```

```
support-led.ads
support-logs.ads
support-rng.ads
support-rtclsi.ads
support-services.ads
```

A later project will utilize these directly - illustrating a technique for reuse.

4.7. Development Insights

Software development for microcontrollers at the low level is all about interfacing with memory mapped features through interface registers. In most environments, a **Hardware Abstraction Layer** protects engineers from having to dig down too deep into the complications of register manipulation. Thus far the previous projectlets we were shielded by the ADL.

© Raja Srinivasan. All rights reserved.

Listing 4.10 Convert cli to a task

File : support-services.adb

```
0009 |    task body cli_Service_Type is
0010 |    begin
0011 |       loop
0012 |          declare
0013 |             Req : String := Receive;
0014 |          begin
0015 |             cmd.Execute( Req );
0016 |          end ;
0017 |       end loop ;
0018 |    end cli_Service_Type;
```

In this projectlet, we utilize the **realtime clock** and build our own HAL. In this effort, the necessary definitions such as register components, bit fields, addresses etc are provided by the ADL in the form of the SVD translations. Most ARM Cortex architecture microcontrollers provide a complete description in this form.

```
STM32_SVD.RCC
STM32_SVD.PWR
STM32_SVD.RTC
```

The above packages enable the proper initialization and setting the date and time and retrieving them from the registers.

© Raja Srinivasan. All rights reserved.

5. Data Integrity

The serial IO with GPIO pins is quite versatile as illustrated in earlier examples - including for communications - be it for a command language interface or for transmission of logs. It is but a small step to leverage this for transmission of sensor data out or setpoint data in - mostly as binary data. As the criticality of the data increases, the need for data integrity increases correspondingly. A fairly simple approach is to use **cyclic redundancy check** with the sender computing a CRC for the data transmitted and the receiver computing the CRC for the data received. When both match, the message can be considered to have been received correctly. Any differences will imply a corruption of some kind and thus the message can be discarded.

Depending on the nature of the data, the ***polynomial*** used in the CRC calculation, it is highly likely to catch any single bit errors. Further analysis of error detection and correction is beyond the scope of this book.

CRC calculations could become compute intensive and such overhead could be minimized using the CRC calculator available in the hardware. This will be the focus of this chapter.

5.1. CRC support

The STM32F4 - used for this book features a CRC peripheral that can be leveraged for communication needs. In this project, an interface is provided for applications as follows:

Listing 5.1 Interface to the CRC peripheral

```
File : support-crc.ads

0008 |    function Calculate ( Data : in Unsigned_32  ) return Unsigned_32 ;
0009 |    function Calculate ( Data : in Block_32 ) return Unsigned_32 ;
0010 |    function Calculate ( Data : in Block_16 ) return Unsigned_32 ;
0011 |    function Calculate ( Data : in Block_8 ) return Unsigned_32 ;
0012 |    function Calculate ( Data : in System.Storage_Elements.Storage_Array )
0013 |                        return Unsigned_32 ;
0014 |    function Calculate ( DataPtr : System.Address ;
0015 |                          Length : System.Storage_Elements.Storage_Offset )
0016 |                        return Unsigned_32 ;
```

Primary hardware input is 32 bit unsigned numbers which is leveraged for other data block lengths as seen above. The adaptation in terms of extending say an 8 bit input into a 32 bit unsigned is left to the application. Little endian, Big endian approaches call for zero padding in different ways. A functional implementation is as shown:

5.1.1. cli session with crc of test vectors

Since we have to ability to incorporate arbitrary cli interactions, a set of test vectors are used to illustrate the support. Of course an implementation on a host is needed typically where the transmitted messages - logs or sensor feedback or whatever are captured so the results can be compared.

```
CRC of
       [ 305419896,  3735928559] is d92b271b
CRC of
       [ 1,  2,  3,  4] is 955ae3fd
d92b271b
955ae3fd
Test Vectors
 1 => c3c5c0cc
 2 => 63375ec3
 3 => 615eab4a
 4 => 1dabe74f
 5 => ba237be3
 6 => 5ff5c445
 7 => 5afe83d0
 8 => a3141bda
cli> help
help    - Display this help message
version - Display version information
random  - Display a random number
date    - Display the date or set it with args: yy mm dd dayname. use . to indicate same
time    - Display the time or set it with args: hh mm ss. use . to indicate same
led     - [blue|green|red|orange|ext] toggles the led
crc     - calculate the crc of test vectors or supplied string
cli> crc "Hello"
3cd1eec9
cli> crc "Ada"
9f64d6df
cli>
```

5.1.2. Application loop

The main loop itself is a trivial application repeating the same CRC calculation ad infinitum:

The output in this case is captured as a **log** on a different interface from the board. A **minicom** session of the log capture is :

```
17:51:14 CRC: The is: 0cbeb623
17:51:16 CRC: quick is: 35243e17
17:51:18 CRC: brown is: 5b50750a
17:51:20 CRC: fox is: f3342c74
17:51:22 CRC: jumps is: 3eb7d624
17:51:24 CRC: over is: fcfca4eb
```

© Raja Srinivasan. All rights reserved.

```
17:51:27 CRC: the is: 949d00c3
17:51:29 CRC: lazy is: 59f8a6e2
17:51:31 CRC: dog is: a4c70a72
17:51:33 CRC: The is: 0cbeb623
17:51:35 CRC: quick is: 35243e17
17:51:37 CRC: brown is: 5b50750a
17:51:39 CRC: fox is: f3342c74
17:51:41 CRC: jumps is: 3eb7d624
17:51:43 CRC: over is: fcfca4eb
17:51:45 CRC: the is: 949d00c3
17:51:47 CRC: lazy is: 59f8a6e2
17:51:49 CRC: dog is: a4c70a72
```

5.2. Implementation

The only additional feature of this project being the CRC computation, a new package **support.crc** is added to our library. However instead of replicating the entire library to this project, the earlier project's source code is directly included as follows:

File : crc.gpr

```
0010 |    for Source_Dirs use ("src/", "config/", "../03_logger/src/"
0011 |                                          , "../03_logger/imports/");
```

In later projects, a full fledged library will be developed.

© Raja Srinivasan. All rights reserved.

Listing 5.2 CRC calculation of an arbitrary length memory block

File : support-crc.adb

```
0066 |    function Calculate ( DataPtr : System.Address ;
0067 |                         Length : System.Storage_Elements.Storage_Offset )
0068 |                         return Unsigned_32 is
0069 |       use System.Storage_Elements ;
0070 |       Data : Storage_Array (1..Length) ;
0071 |       for Data'Address use DataPtr ;
0072 |
0073 |       use System.Storage_Elements ;
0074 |       longwords : System.Storage_Elements.Storage_Offset := Length / 4;
0075 |       byte : System.Storage_Elements.Storage_Offset ;
0076 |       lastword : Unsigned_32 := 0;
0077 |       Shift_Count : Integer := 0;
0078 |    begin
0079 |       Reset_Calculator;
0080 |       for i in 1..longwords loop
0081 |          byte := (i-1)*4 + 1 ;
0082 |          Feed_Word ( Shift_Left(Unsigned_32(Data(byte+3)),24) +
0083 |                      Shift_Left(Unsigned_32(Data(byte+2)),16) +
0084 |                      Shift_Left(Unsigned_32(Data(byte+1)),8) +
0085 |                      Unsigned_32(Data(byte)) ) ;
0086 |       end loop;
0087 |       if longwords * 4 < Data'Length then
0088 |          for i in  longwords*4 + 1 .. Data'Last loop
0089 |             if Shift_Count > 0 then
0090 |                lastword := Shift_Left( Unsigned_32(Data(i)) ,
0091 |                                        Shift_Count )
0092 |                                        + lastword ;
0093 |             else
0094 |                lastword := Unsigned_32(Data(i)) ;
0095 |             end if ;
0096 |             Shift_Count := Shift_Count + 8 ;
0097 |          end loop;
0098 |          Feed_Word (lastword);
0099 |       end if ;
0100 |
0101 |       return Current_Value;
0102 |    end Calculate ;
```

© Raja Srinivasan. All rights reserved.

Listing 5.3 Main loop repeating for ever

File : crc.adb

```
0034 |    loop
0035 |       support.logs.Put( support.rtclsi.Get_Time & " ") ;
0036 |       while wordend < test'Last and test(wordend) /= ' ' loop
0037 |          wordend := wordend + 1 ;
0038 |       end loop ;
0039 |
0040 |       if wordend = test'Last then
0041 |          support.logs.Put( "CRC: " & test(wordbegin .. wordend) & " is: " );
0042 |       else
0043 |          support.logs.Put( "CRC: " & test(wordbegin .. wordend - 1) & " is: " );
0044 |       end if ;
0045 |
0046 |       crc_value := support.crc.Calculate( test(wordbegin)'Address , Storage_Offset(wo
0047 |       support.logs.Put_Line( hex.Image( crc_value ) ) ;
0048 |
0049 |       if wordend >= test'Last then
0050 |          wordbegin := test'First ;
0051 |       else
0052 |          wordbegin := wordend + 1 ;
0053 |       end if ;
0054 |       wordend := wordbegin ;
0055 |       delay 2.0 ;
0056 |    end loop ;
```

© Raja Srinivasan. All rights reserved.

6. Library Organization

```
This chapter is based on the projects:

- 04a_crc
- 00_toolkit
```

6.1. Projectlet Goals

Goals of this projectlet are:

- Replicate the **crc** project
- Evolve an architecture of a toolkit that can be the platform for all other projects. In fact this layer should be able to serve as the starting point for a fullblown application with a few cooperating applications.
- Expectation is for the addition of standard features such as application health annunciation through an LED, a log, a terminal like command line interface to be included transparently in the application without major efforts - excepting hardware dependencies.

6.2. Organization of the codebase

We take a breather now to revise our code organization for enabling quick composition of new projects. In particular an approach to a library of modules to support common functions began to evolve in the previous projects. Based on our experiments, in this projectlet, the design of a **toolkit** is attempted. In particular an application oriented library layer **toolkit** is introduced to shield applications somewhat from having to reengineer many standard support functions.

In particular the earlier project **crc** layered on top of such a toolkit simply includes the library in its project file and application itself is very simple problem oriented code. The toolkit is included as:

```
File : crc.gpr

0002 | with "../00_toolkit/toolkit.gpr";
0003 | with "config/crc_config.gpr";
0004 | with "../common.gpr" ;
0005 |
0006 | project Crc is
0007 |
0008 |    for Target use "arm-eabi" ;
0009 |    for Runtime ("Ada") use "embedded-stm32f4" ;
```

As an aside, the above fragment illustrates, an abstract project file **common.gpr** is designed for inclusion into other projects - in order to standardize compiler switches and the like:

```
File : crc.gpr

0019 |    package Compiler is
0020 |       for Default_Switches ("Ada") use common.Compiler_Switches ;
0021 |    end Compiler;
```

6.3. Simplified application

The toolkit does not have a great impact on the application code as shown:

Listing 6.1 Leveraging the toolkit

```
File : crc.adb

0039 |    loop
0040 |       while wordend < test'Last and test(wordend) /= ' ' loop
0041 |          wordend := wordend + 1 ;
0042 |       end loop ;
0043 |       crc_value := toolkit.crc.Calculate( test(wordbegin)'Address
0044 |                                          , wordend - wordbegin ) ;
0045 |       if wordend = test'Last then
0046 |          toolkit.logs.logger.Put_Line( test(wordbegin .. wordend) &
0047 |                                        " is: " & hex.Image(crc_value)
0048 |                                        , source => myname ) ;
0049 |       else
0050 |          toolkit.logs.logger.Put_Line( test(wordbegin .. wordend - 1) &
0051 |                                        " is: " & hex.Image(crc_value)
0052 |                                      , source => myname ) ;
0053 |       end if ;
0054 |
0055 |       if wordend >= test'Last then
0056 |          crc_value := toolkit.crc.Calculate( test(test'First)'Address
0057 |                                             , test'Length ) ;
0058 |          toolkit.logs.logger.Put_Line ( "CRC(" & test & ") is: "
0059 |                                          & hex.Image(crc_value)
0060 |                                       , level => toolkit.logs.WARNING
0061 |                                       , source => myname ) ;
0062 |          wordbegin := test'First ;
0063 |       else
0064 |          wordbegin := wordend + 1 ;
0065 |       end if ;
0066 |       wordend := wordbegin ;
0067 |       delay 2.0 ;
0068 |    end loop ;
```

© Raja Srinivasan. All rights reserved.

The logs should now be a bit more fanciful, with automatic timestamps, logging level support and a way to identify the ***source*** of a log line. The entire log channel now can support multiple generator.

The library is incorporated in the application as:

Listing 6.2 Application usage of the library

```
File : crc.adb

0008 | with hex ;
0009 | with toolkit.led ;
0010 | pragma Elaborate( toolkit.led ) ;
0011 |
0012 | with toolkit.services;
0013 | pragma Elaborate( toolkit.services ) ;
0014 |
0015 | with toolkit.rtclsi ;
0016 | with toolkit.logs ;
0017 | with toolkit.crc ;
```

All of this has a very familiar feel though the target environment is an embedded bare metal platform. It will be a challenge to use an entirely different board without changing the application code - which will be addressed in the next book in this series.

6.3.1. Example run

When this application is run, several features are illustrated - an LED blinks, the logs are richer, the log level (I for informational and W for warning) as described in **toolkit.logs** and source (Crc) are clearly identified:

```
17:50:16 -W- [Crc...] : CRC(The quick brown fox jumps over the lazy dog) is: 1ce6cfb1
17:50:18 -I- [Crc...] : The is: 0cbeb623
17:50:20 -I- [Crc...] : quick is: 35243e17
17:50:22 -I- [Crc...] : brown is: 5b50750a
17:50:24 -I- [Crc...] : fox is: f3342c74
17:50:26 -I- [Crc...] : jumps is: 3eb7d624
17:50:28 -I- [Crc...] : over is: fcfca4eb
17:50:30 -I- [Crc...] : the is: 949d00c3
17:50:33 -I- [Crc...] : lazy is: 59f8a6e2
17:50:35 -I- [Crc...] : dog is: a4c70a72
17:50:35 -W- [Crc...] : CRC(The quick brown fox jumps over the lazy dog) is: 1ce6cfb1
17:50:37 -I- [Crc...] : The is: 0cbeb623
17:50:39 -I- [Crc...] : quick is: 35243e17
17:50:41 -I- [Crc...] : brown is: 5b50750a
17:50:43 -I- [Crc...] : fox is: f3342c74
17:50:45 -I- [Crc...] : jumps is: 3eb7d624
```

In addition, with very little additional actions, a **terminal** is automatically created and is operational. A session that exercises the CRC from the terminal:

© Raja Srinivasan. All rights reserved.

```
led      - [blue|green|red|orange|ext] toggles the led
crc      - hex string or quoted string to compute and display CRC
cli> crc aabbccdd
19354eb8
eb863705 "AdaCore"

CRC of c
       [ 305419896,  3735928559] is: d92b271b
Calculated as a block is: d92b271b
CRC of
       [ 1,  2,  3,  4] is: 955ae3fd
Calculated as a block is: 955ae3fd
CRC of  1 bytes is: c3c5c0cc
CRC of  2 bytes is: 63375ec3
CRC of  3 bytes is: 615eab4a
CRC of  4 bytes is: 1dabe74f
CRC of  5 bytes is: ba237be3
CRC of  6 bytes is: 5ff5c445
CRC of  7 bytes is: 5afe83d0
CRC of  8 bytes is: a3141bda
cli>
```

Of course, the commands are implemented as part of the toolkit but in later projects a more application oriented command line support will be illustred.

6.4. Architecture of the library

A library for the embedded environment is quite similar to one for general purpose environments. In this effort, the library consists of several functions imported from the ADL such as **serial_io** and its supporting functions such as **message_buffer**s. These are included in the ADL as examples and not as part of the library itself.

The **toolkit** from the earlier work contributes a package such as **hex** but customized for the embedded platform.

Beyond that all the disparate **support** packages from earlier projects are replicated here under the package **toolkit** in the process making them adaptible to applications easily. All the support that may be hardware related are all collected under a package tree. To summarize the library features the following package tree:

Package	Function
toolkit	toplevel
toolkit.button	Pushbuttons
toolkit.cli	command language
toolkit.cli.cmd	commands supported
toolkit.crc	CRC
toolkit.led	LEDs
toolkit.led.health	App health
toolkit.logs	Text logs
toolkit.rng	Random numbers

© Raja Srinivasan. All rights reserved.

Package	Function
toolkit.rtclsi	Realtime clock
toolkit.services	a way to initiate services

The application crc is an example of applying this as is the **button** projectlet to follow.

The library project is specified similar to the ones for general purpose platforms:

```
File : toolkit.gpr

0002 | with "../../adl/boards/stm32f407_discovery/stm32f407_discovery_full.gpr";
0003 | with "config/toolkit_config.gpr";
0004 | with "../common.gpr" ;
0005 | project Toolkit is
0006 |
0007 |    for Target use "arm-eabi" ;
0008 |    for Runtime ("Ada") use "embedded-stm32f4" ;
0009 |
0010 |    for Library_Name use "Toolkit";
0011 |    for Library_Version use Project'Library_Name & ".so." & Toolkit_Config.Crate_Vers
0012 |
0013 |    type Library_Type_Type is ("relocatable", "static", "static-pic");
0014 |    Library_Type : Library_Type_Type :=
0015 |      external ("TOOLKIT_LIBRARY_TYPE", external ("LIBRARY_TYPE", "static"));
0016 |    for Library_Kind use Library_Type;
```

Things to note in this project file are:

- The inclusion of the ADL which is done in this one location and does not have to be repeated in each project.
- By default a static library is created
- The library name is specified as **Toolkit** resulting in libToolkit.a
- A method to have common switches is used - if needed

Most devices (medical or otherwise) will feature multiple low level application modules, of the same architecture and will benefit from this approach of separating the library. The conventional notion of **hardware abstraction layer** HAL is extremely helpful in isolating low level details but the toolkit layer goes further.

6.4.1. Example: Log service

In particular standard constructs such as **tasks**, **protected objects** can be implemented in this layer to help designers to focus on the application. The logging support provided by the toolkit **toolkit.logs** is a pattern that will be replicated many times. From the application's standpoint, a protected object is the destination and source as needed:

In the above, the application uses the **Put_Line** to request a log line but this request is processed only when exclusive access can be granted to the requestor. Only the actual log text generator needs to retrieve these requests using the **Get** interface.

The log text generator itself is a task:

© Raja Srinivasan. All rights reserved.

Listing 6.3 Logging interface

```
File : toolkit-logs.ads

0018 |    procedure Put_Line (This : String);
0019 |
0020 |    protected Logger
0021 |
0022 |      with Priority => System.Priority'Last
0023 |    is
0024 |       -- Application interface. Enables different tasks to send logs without
0025 |       -- interfering with each other. The entry serializes the access to this
0026 |       -- object or facility
0027 |       entry Put_Line
0028 |         (this     : String;
0029 |          level    : message_level_type := INFORMATIONAL;
0030 |          source   : Source_Name_Type := Default_Source_Name;
0031 |          fieldsep : Character := ' ');
0032 |
0033 |       -- Used only by the owner of the channel ie the log record generator.
0034 |       -- Still need to coordiate access to the channel. The entry ensures that
0035 |       entry Get
0036 |         (msg    : in out Message_Buffers.Message;
0037 |          level  : out message_level_type;
0038 |          source : out Source_Name_Type);
0039 |
0040 |    private
0041 |       initialized : Boolean := False;
```

This task has exclusive control of the hardware and leverages the **non blocking** interfaces to the **serial_io** ports. The specifics of the GPIO pins, serial discipline (e.g. 8 bits, 1 stop bit, 115200 baud) are all hidden from the application. In fact the actual text that is generated contains the applications text but additional information such as timestamps are added transparently.

A particular enhancement that will be useful but not implemented is the addition of a CRC to the message. For a log text, it **may** be considered an overhead but in later projectlets, when we apply this pattern to binary data - including sensor feedback and setpoints, such data protection will be mandatory.

© Raja Srinivasan. All rights reserved.

Listing 6.4 Task to generate the log text

```
File : toolkit-logs.adb

0118 |    task body Service_Type is
0119 |       msg : Message_Buffers.Message (Physical_Size => 160);  -- arbitrary size
0120 |       ml  : message_level_type;
0121 |       src : Source_Name_Type;
0122 |    begin
0123 |       loop
0124 |          Logger.Get (msg, ml, src);
0125 |          Put_Line
0126 |            (rtclsi.Get_Time
0127 |             & " "
0128 |             & Separator
0129 |             & " "
0130 |             & Image (ml)
0131 |             & " "
0132 |             & Separator
0133 |             & " ["
0134 |             & src
0135 |             & "] "
0136 |             & Separator
0137 |             & msg.Content);
0138 |       end loop;
0139 |    end Service_Type;
```

© Raja Srinivasan. All rights reserved.

7. Pushbutton

```
This chapter is based on the projectlet:

- 05_button
```

7.1. Projectlet Goals

Goals of this projectlet are:

- Demonstrate the application of the **toolkit** library
- Support the onboard pushbutton
- Support an external pushbutton - which will require specifying and initializing the hardware
- Reflect the state of the pushbutton (pushed/released) in setting the LED On or Off

7.2. Dual applications

This projectlet delivers the solutions in two different applications **button** for the onboard pushbutton and **button_ext** for an external pushbutton. The significant difference between the two are the hardware specification and the initialization required. The external application pushbutton is mapped to a particular GPIO pin. In addition the interface is configured to match the design of the pushbutton itself.

Listing 7.1 Connect and initialize external pushbutton

```
File : toolkit-button.adb

0005 |    procedure Connect_Ext_Button (point : GPIO_Point := Ext_Button_Point) is
0006 |    begin
0007 |       Enable_Clock (point);
0008 |       Configure_IO (point, (Mode_In, Resistors => Pull_Up));
0009 |    end Connect_Ext_Button;
```

As indicated, the clock is enabled explicitly for this interface, and further configuration is required. While the mode **Input** is expected the **pull_up resistor** specification which depends on the pushbutton design and the available onboard digital circuitry is more involved and is outside the scope of this book.

The application itself is demonstrative:

The button is monitored at 10Hz reflecting the state setting or clearing the red LED. Clearly the LED hardware has not been abstracted out but easily done. In the operation itself however,

Listing 7.2 External button application

```
File : button_ext.adb

0009 | procedure Button_ext is
0010 | begin
0011 |    stm32.gpio.Clear( stm32.board.Red_LED ) ;
0012 |    toolkit.button.Connect_Ext_Button ;
0013 |    loop
0014 |       if stm32.gpio.Set(toolkit.button.Ext_Button_Point) then
0015 |          stm32.gpio.Clear( stm32.board.Red_LED ) ;
0016 |       else
0017 |          stm32.gpio.Set( stm32.board.Red_LED ) ;
0018 |       end if ;
0019 |       delay 0.1 ;
0020 |    end loop ;
0021 | end Button_ext;
```

the application **health** LED is included automatically and the green LED blinks independent of the main loop which monitors the button state.

© Raja Srinivasan. All rights reserved.

8. Analog world

```
This chapter is based on the projectlet:

- 06_pot
```

8.1. Projectlet Goals

We leave the **comfortable** digital world and wander into **analog** territory in this projectlet, particularly analog to digital conversion. Our board features 3 distinct **Analog to Digital Converter**s. We start our exploration of ADCs with the onboard analog sensors that are wired to be digitized in the application **analogob** :

- VBat - Battery voltage. In the discovery board used in these examples, there is no battery and instead wired to the 5V rail.
- Temperature - Internal temperature. Not to be confused with the board temperature this is an internal die temperature!

Leveraging the ability to digitize input from GPIO pins, two different applications are also included.

batt - for battery testing - coin cells and other batteries to see the voltage they actually deliver. For a given battery of course the voltage should be relatively stable though ***leaking*** over long periods.

pot - a **potentiometer** or a voltage divider which can be manually adjusted to see varying voltage outputs.

8.1.1. Setup the conversions

Typical ADCs including ours are quite configurable. In these projects the following conservative configuration is used:

Listing 8.1 Setup the ADC component

```
File : toolkit-adc.adb

0160 |    STM32.Device.Reset_All_ADC_Units;
0161 |    STM32.Device.Enable_Clock (stm32.Device.ADC_1);
0162 |    STM32.ADC.Enable (stm32.Device.ADC_1);
0163 |    STM32.ADC.Configure_Unit
0164 |      (STM32.Device.ADC_1,
0165 |       stm32.ADC.ADC_Resolution_12_Bits,
0166 |       stm32.ADC.Right_Aligned);
```

ADC_1 component is first enabled and then configured. We are asking for 12 bits conversion to be delivered **Right_Aligned** which is another way of indicating **Little Endian**.

Needless to say the 12 bits of resolution is highly application dependent and definitely overkill for the simple applications in this projectlet.

Listing 8.2 Setup the ADC channels

```
File : toolkit-adc.adb

0013 |    procedure Initialize_Onboard_Channels is
0014 |       use stm32.ADC;
0015 |       conversions : Regular_Channel_Conversions :=
0016 |         ((VBat_Channel, Sample_480_Cycles),
0017 |          (stm32.Device.Temperature_Channel, Sample_480_Cycles));
0018 |
0019 |    begin
0020 |
0021 |       Configure_Regular_Conversions
0022 |         (STM32.Device.ADC_1, False, Software_Triggered, False, conversions);
0023 |
0024 |    end Initialize_Onboard_Channels;
```

The fragment above setups the ADC to convert 2 analog values the VBat and Temperature - each indicated by their channel number. Specifically we budget a conservative 480 Cycles to digitize these quantities. Once again, depending on the input stage, a lower number of cycles may well be adequate. This of course determines the time required for the conversion to be complete hence impacting the throughput.

8.1.2. Interpreting the ADC output

The ADC outputs an integer - of width 12 bits in the above examples - which requires a translation to the physical units. In the case of VBat, we are looking to translate the results into a voltage whereas in the case of the temperature a translation to Centigrade or Fahrenheit is desired. The conversion is a fairly straight forward linear interpolation:

The fragment above shows a few assumptions such as what is the **reference** voltage, minimum and maximum voltage levels of the input and so on. In particular for VBat, the internal details of the board design are incorporated in this calculation.

Temperature measurements require a few assumptions similar to the above. Prior calibration yields specific mapping of temperatures to raw counts and provides the interpolation equations.

8.1.3. Measurement Accuracy

Precision and accuracy are critical parameters to understand in the conversion process. The linear interpolation of the digital output resulting in a **floating point** result is a little misleading. As shown in the following fragment **Temperature_Type** is declared to align better with the measurement results. This **fixed** data type - unique feature of **Ada** used appropriately in all the related algorithms will lead to reliable results.

© Raja Srinivasan. All rights reserved.

Listing 8.3 Raw counts conversion - Voltages

File : toolkit-adc.adb

```
0069 |    function Raw_To_Volts (Raw : Unsigned_16) return Voltage_Type is
0070 |       V_Ref : constant Float := 3.3;
0071 |    begin
0072 |       return Voltage_Type ((Float (Raw) / 4095.0) * V_Ref);
0073 |    end Raw_To_Volts;
0074 |
0075 |    function Raw_To_Volts_Vbat (Raw : Unsigned_16) return Voltage_Type is
0076 |       V_Ref        : constant Float := 3.3;
0077 |       VBat_Divisor : constant Float := 4.0; -- internal divider
0078 |    begin
0079 |       return Voltage_Type ((Float (Raw) / 4095.0) * V_Ref * VBat_Divisor);
0080 |    end Raw_To_Volts_VBat;
```

Listing 8.4 Raw counts Temperature - deg C

File : toolkit-adc.adb

```
0098 |    function Raw_To_Celsius (Raw : Unsigned_16) return Temperature_Type is
0099 |       V_Ref     : constant Float := 3.3;
0100 |       V_25      : constant Float := 0.760;   -- volts at 25 Ã Â°C
0101 |       Avg_Slope : constant Float := 0.0025;  -- 2.5 mV/Ã Â°C
0102 |       V_Sense   : Float;
0103 |    begin
0104 |       V_Sense := (Float (Raw) / 4095.0) * V_Ref;
0105 |       return Temperature_Type (((V_Sense - V_25) / Avg_Slope) + 25.0);
0106 |    end Raw_To_Celsius;
```

Floating point operations are quite expensive in terms of cycles and has not been an inherent feature of embedded microcontrollers till recently. Still it might be wise to minimize reliance on floating point - in particular where convergence of algorithms is critical. Fixed point whereby internall computations are done in Integer should be evaluated in such contexts. Support for fixed point is available natively in **Ada**.

8.1.4. Experimental Setup and Results

8.1.4.1. Onboard sensors - VBat & Temperature

These require no physical setup. At a desired periodicity, the ADC is **read** for the channels which were setup, in the original order:

The ADL does provide abstractions for most of the ADC operation; still need access to some configuration registers made accessible by **STM32.SVD_ADC**. The specifics are peculiar to the STM32 and outside the scope of this discussion. Reading of the VBat and VTemp have to be done in sequence since while the ADC was setup we provided 2 connections in that order.

© Raja Srinivasan. All rights reserved.

Listing 8.5 Match the measurement accuracy

```
File : toolkit-adc.ads

0017 |    type Temperature_Type is delta 0.1 digits 3;
0018 |    function Raw_To_Celsius (Raw : Unsigned_16) return Temperature_Type;
```

Listing 8.6 Read the temperature

```
File : toolkit-adc.adb

0045 |    procedure Read_Temp (Temp : out Unsigned_16) is
0046 |       use stm32.ADC;
0047 |    begin
0048 |       STM32_SVD.ADC.C_ADC_Periph.CCR.VBATE := False;
0049 |       STM32_SVD.ADC.C_ADC_Periph.CCR.TSVREFE := True;
0050 |
0051 |       Clear_Status (STM32.Device.ADC_1, Regular_Channel_Conversion_Complete);
0052 |       Start_Conversion (STM32.Device.ADC_1);
0053 |       while not Status
0054 |                   (STM32.Device.ADC_1, Regular_Channel_Conversion_Complete)
0055 |       loop
0056 |          null;
0057 |       end loop;
0058 |       Temp := Unsigned_16 (Conversion_Value (STM32.Device.ADC_1));
0059 |    end Read_Temp;
0060 |
0061 |    procedure Read (VBat : out Unsigned_16; Temp : out Unsigned_16) is
0062 |    begin
0063 |       Read_VBat (VBat);
0064 |       Read_Temp (Temp);
0065 |    end Read;
```

```
00:14:50 -I- [sensor] :  VBat=  1390 VBatF=  4.4 Temp=  1056 TempF=  61.3
00:15:00 -I- [sensor] :  VBat=  1418 VBatF=  4.5 Temp=  1058 TempF=  62.0
00:15:11 -I- [sensor] :  VBat=  1389 VBatF=  4.4 Temp=  1058 TempF=  62.0
00:15:21 -I- [sensor] :  VBat=  1396 VBatF=  4.4 Temp=  1058 TempF=  62.0
00:15:31 -I- [sensor] :  VBat=  1394 VBatF=  4.4 Temp=  1062 TempF=  63.3
00:15:42 -I- [sensor] :  VBat=  1392 VBatF=  4.4 Temp=  1060 TempF=  62.6
00:15:52 -I- [sensor] :  VBat=  1395 VBatF=  4.4 Temp=  1056 TempF=  61.3
00:16:02 -I- [sensor] :  VBat=  1414 VBatF=  4.5 Temp=  1062 TempF=  63.3
```

The above log is a fragment generated by **sensorob**.

8.1.4.2. Battery Testing

Battery voltage testing requires a physical setup and a GPIO pin for this function.

© Raja Srinivasan. All rights reserved.

Figure 8.1.: AA battery

The battery voltage may be more or less stable but over a long time, the terminal voltage will be lower and will probably eventually go to 0 Volts.

```
amazon basics AA battery (Battery daddy setup)
01:01:39 -I- [batt  ] :  batv=  2068 batvF=  1.6
01:01:40 -I- [batt  ] :  batv=  2074 batvF=  1.6
01:01:41 -I- [batt  ] :  batv=  2074 batvF=  1.6
01:01:42 -I- [batt  ] :  batv=  2072 batvF=  1.6
01:01:43 -I- [batt  ] :  batv=  2080 batvF=  1.6
01:01:44 -I- [batt  ] :  batv=  2072 batvF=  1.6
01:01:45 -I- [batt  ] :  batv=  2066 batvF=  1.6
01:01:46 -I- [batt  ] :  batv=  2076 batvF=  1.6
01:01:47 -I- [batt  ] :  batv=  2074 batvF=  1.6
01:01:48 -I- [batt  ] :  batv=  2066 batvF=  1.6
```

The voltage while stable is misleading. The battery holder arrangement was not the most robust. No factory calibration of the ADC was used in the above. The assumptions we made to compute the interpolation equation are also somewhat arbitrary and approximate.

8.1.4.3. Potentiometer

A potentiometer is a little more interesting. With the ability to move the tap, it is easy to see the ADC responding with a range of values - not just a static value.

© Raja Srinivasan. All rights reserved.

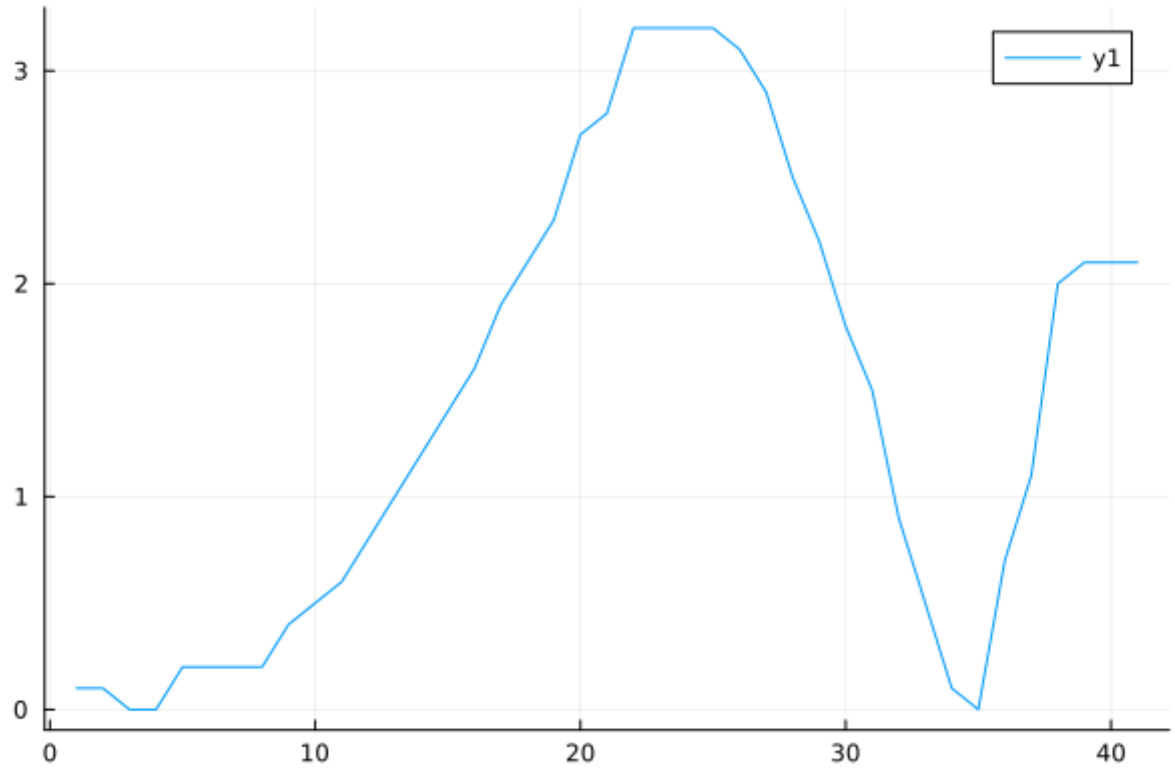


Figure 8.2.: Potentiometer

8.1.4.4. Fader

Even more interesting is to generate a varying voltage to feed into the ADC. To achieve this, it is straightforward to use an **Arduino UNO**. In this case we adapt the ***fader*** example adding a capacitor to smooth out the variations - thus generating a stable voltage. The ADC pin is now connected to this output.

© Raja Srinivasan. All rights reserved.

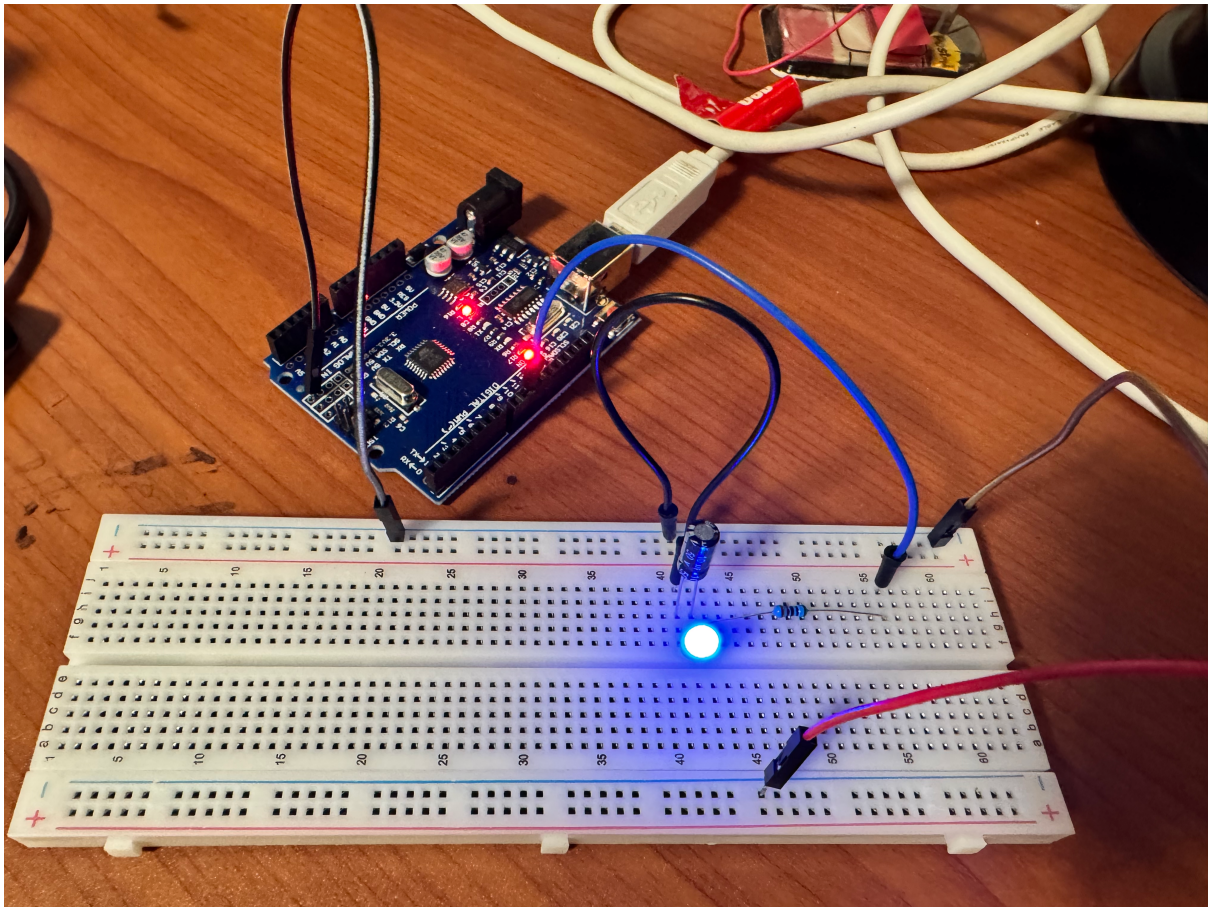


Figure 8.3.: Fader driver

And now the application reads a varying voltage and plotting the voltages yields:

© Raja Srinivasan. All rights reserved.

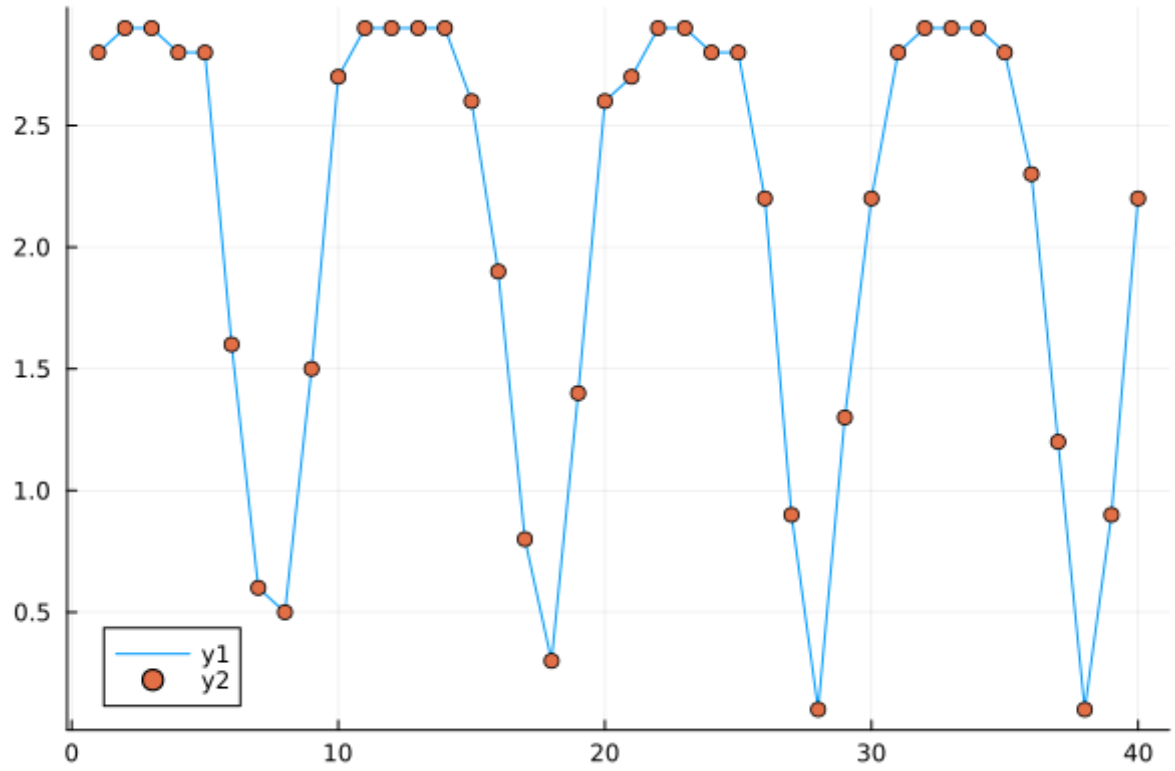


Figure 8.4.: LED Fader

Please note that the fader operates with 30 millisecond gap while the digitizer operates at 1 Hz.

8.2. Development Insights

Analog to Digital conversion is perhaps the most critical component of any embedded system. The challenge of dealing with the physics (or any other natural phenomena) starts with sensing and converting the result to a digital form. While most development kits incorporate an ADC, production devices will need a higher quality converter. Sampling rate, compensation for ambient temperature, dealing with supply voltage jitter are some of the challenges that will determine the success of the device - leading to the choice of specialized ADCs.

Sampling the ambient room temperature for example may be reasonable at 1Hz or even slower for a residential thermostat. An EKG on the other hand will benefit from higher sample rates say 1KHz in order to truly represent the physiology. Similar arguments may lead to a 24 bit width digitization for the latter while 8 bits may be sufficient for the temperature.

It is also likely that an external ADC will provide an I2C or SPI interface leaving the microcontroller free from the numerous low level details.

© Raja Srinivasan. All rights reserved.

9. Interrupts and Timers

```
This chapter is based on the projectlet:

- 07_timers
```

9.1. Projectlet Goals

While time and its passage has been at the core of all the projects thus far, we took the easy route by and large depending primarily on the builtin **delay** to provide the desired timing. In addition we activated the onboard **real time clock** (RTC) to maintain an independent (from the run time libraries of **Ada**) clock.

The logfiles generated revealed that the notion of time, clock are rather tricky - dependent as they are on the hardware availability and their accuracy thereof. The hardware used in these efforts namely the STM32F4 Discovery board is a development tool and cannot be expected to feature super high accuracy as would be required in a production environment - particularly safety critical applications.

In addition, the application design that relies on the delay statement operates on a cadence that is not too predictable. At the minimum, the cost of the operation being performed for example digitizing an analog value as we observed in other projects - may be a reasonable fraction of the delay time and thus the cadence could be significantly different from the expectation. In addition the application load on the microcontroller may well have an impact on this as well. The situation will be **unacceptable** in a **hard real time system**.

In this projectlet, we explore the built in hardware timers. Supposedly these timers are highly accurate and we leverage **Interrupts** generated by the timers for the periodicity of application functions. Leveraging the logging support from the toolkit which in turn relies on the RTC, we can compare the performance of the clocks.

Also explored is the pwm mode of the timers. Some timers eg. Timer_3 and Timer_8 have this feature and this is applied to *fade*ing an external LED.

9.1.1. Basic Approach

The Discovery board features quite a few timers. For this exploration we depend on the simplest of them ***Timer_6*** and ***Timer_7***. The timers can generate Interrupts at a desired rate and not much more. One of these timers we will use to just maintain a counter (1 second) and the other will be used for Analog to Digital conversion of onboard sensors.

The digitized values and the counter itself will be logged which can be analyzed to understand the drift.

9.2. Implementation

9.2.1. Toolkit Support

Timers are of course very dependent on the hardware - the microcontroller architecture and the board design. The package **toolkit.timers** is designed for the discovery board we are using in this set of projectlets. This pattern will lend itself and has to be adapted to the specific hardware.

Listing 9.1 Basic Timers

```
File : toolkit-timers.ads

0009 |
0010 |    -- Following provide a very basic timer on the STM32F4 Discovery board
0011 |    -- Timerclock: 84 MHz
0012 |    -- 16 bit Prescaler and Reload Counters
0013 |    basic_1 : constant access stm32.timers.Timer := stm32.Device.Timer_6'access;
0014 |    basic_2 : constant access stm32.timers.Timer := stm32.Device.Timer_7'access;
0015 |
0016 |    type UserCallback is access procedure (timerno : Integer);
0017 |
0018 |    -- Some combination of Prescaler and the Counter can result in
0019 |    -- different cadences
0020 |    type cadence_type is (One_Hz, One_KHz, One_Fourth_Hz);
0021 |
0022 |    procedure SetupBasic
0023 |      (tmr     : access stm32.timers.Timer;
0024 |       cb      : UserCallback;
0025 |       cadence : cadence_type := One_Hz);
0026 |
```

In this projectlet, we use the 2 most basic timers on our circuit board. The initializations required are very simple:

The toolkit provides an intermediary ISR in order to take care of required housekeeping such as dismissing the interrupt and then the application callback is invoked:

9.2.2. Interfacing the Interrupts and application

Interrupts are serviced by **Interrupt Service Routine** (ISR)s. In other words, the application installs an ISR (handled in our design by the toolkit) which is invoked when the interrupt is generated. The ISR in turn needs to perform the function needed by the application. The ISR is required to be fast and does not have the flexibility to perform lengthy operations. For the specific timers used in this projectlet, 2 **callback**s are used.

The above specifies the two ISRs. The ***LED_Toggle_Callback*** will as its name suggests toggle an LED which can be expected to be a fast operation.

© Raja Srinivasan. All rights reserved.

Listing 9.2 Initialization - basic timers

```
File : toolkit-timers.adb

0049 |       STM32.Device.Enable_Clock (tmr.all);
0050 |       case cadence is
0051 |          when One_Hz =>
0052 |             stm32.Timers.Configure
0053 |               (tmr.all, Prescaler => 8399, Period => 9999);
0054 |
0055 |          when One_KHz =>
0056 |             stm32.Timers.Configure
0057 |               (tmr.all, Prescaler => 16799, Period => 499);
0058 |
0059 |          when One_Fourth_Hz =>
0060 |             stm32.Timers.Configure
0061 |               (tmr.all, Prescaler => 41999, Period => 7999);
0062 |       end case;
0063 |       stm32.timers.Set_Autoreload_Preload (tmr.all, true);
0064 |       stm32.timers.Enable_Interrupt
0065 |         (tmr.all, stm32.timers.Timer_Update_Interrupt);
0066 |       stm32.timers.Enable (tmr.all);
```

If the application needs to perform lengthy operations based on the (timer) interrupt then the ISR will have to delegate it to a different task which is illustrated in the ***SensorsOnboard_Callback*** above. In this case a **Suspension_Object** is provided so trigger the operations.

In the case of the LED_Toggle_Callback, the ISR performs only an increment of a counter after toggling the LED. An independent task logs the counter without interfering with the ISR thus removing the lengthy logging operation from the ISR.

In the case of the SensorsOnboard_Callback the ISR will just indicate that the interrupt was received using the Synchronization_Object. This is expected to be a fast operation.

The application will wait on the Suspension_Object to perform the (lengthy) operation such as ADC as illustrated above. In addition to conversion, the data is sent to a **log** which is also somewhat lengthy. With this synchronization method there are no **delay**s needed.

As illustrated, the duty cycle is independently modified by an ISR attached to a different timer. Periodically the PWM timer's duty cycle is changed thus resulting in the LED brightness changing.

9.2.3. Pulse width modulation and fading the LED

The projectlet **pwmfader** sets up a timer to generate square waves. Blinking an LED essentially is achieved by applying a voltage to the LED; when it is **LOW** the LED being off. By verying the duty cycle the effective voltage applied to the LED is varied and thus the brightness of the light emitted can be varied.

© Raja Srinivasan. All rights reserved.

Listing 9.3 Interrupt Service Routine

File : toolkit-timers.adb

```
0025 |       procedure ISR7 is
0026 |       begin
0027 |          -- Required to clear the interrupt state with minimal delay.
0028 |          -- The user callback can focus on its own functions
0029 |          STM32.Timers.Clear_Pending_Interrupt
0030 |            (STM32.Device.Timer_7, STM32.Timers.Timer_Update_Interrupt);
0031 |          -- invoke user callback. The callback should also be extremely fast
0032 |          ucb_isr7.all (7);
0033 |       end ISR7;
```

Listing 9.4 Callback Interface

File : cb.ads

```
0006 |    LED_Toggle_Count : Integer := 0;
0007 |    procedure LED_Toggle_Callback (timerno : Integer);
0008 |
0009 |    subtype DutyCycleRange is Integer range 75 .. 95;
0010 |    dutycycle : Integer := DutyCycleRange'first;
0011 |
0012 |    procedure SetDutyCycle_Callback (timerno : Integer);
0013 |
0014 |    TimeToReadSensors : Ada.Synchronous_Task_Control.Suspension_Object;
0015 |    procedure SensorsOnboard_Callback (timerno : Integer);
0016 |
```

9.2.4. Log generation

We had briefly reviewed the logging support in an earlier chapter. To summarize, from the microcontroller, an outgoing log channel can be established which can be handled by a host using a serial communication discipline. In a typical device, there are multiple independent **task**s each performing a distinct operation. For example, an application may feature a task (thread) to: read a temperature sensor, read an ambient light intensity sensor and another to keep a set of annunciators to indicate the state of the application.

Services that are available at the controller level will then be shared among the different tasks; logging being such a service. Physically the logging service might utilize a Serial port at the low level, transmitting and receiving octets. Different tasks then will queue their log message to a central resource **protected** in Ada terminology to serialize access to this channel. Another task controlling the serial port will then dequeue the messages and have them transmitted.

The **cli** service discussed earlier performed the inverse namely received octets on the serial port and executed the recognized command lines so received.

The key insight we gain is a design to implement shared resources.

© Raja Srinivasan. All rights reserved.

Listing 9.5 Report ticker value

```
File : cb.adb

0025 |       loop
0026 |          delay 1.0;
0027 |          if oldcounter /= LED_Toggle_Count then
0028 |             oldcounter := LED_Toggle_Count;
0029 |             toolkit.logs.logger.Put_Line
0030 |               ("Ticker " & oldcounter'Image, source => myname);
0031 |          end if;
0032 |       end loop;
```

Listing 9.6 Eventflag interface

```
File : cb.adb

0055 |    procedure SensorsOnboard_Callback (timerno : Integer) is
0056 |    begin
0057 |       if timerno = 6 then
0058 |          stm32.Board.Toggle (stm32.Board.Red_LED);
0059 |          Ada.Synchronous_Task_Control.Set_True (TimeToReadSensors);
0060 |       end if;
0061 |    end SensorsOnboard_Callback;
```

9.3. Results

Sample logfile from the application **sensorsob** after almost a day:

```
23:20:47   -I-   [cb    ]  Ticker  81136
23:20:48   -I-   [sensor]   VBat=  1395 VBatF=  4.4 Temp=  1062 TempF=  63.3
23:20:48   -I-   [cb    ]  Ticker  81137
23:20:49   -I-   [cb    ]  Ticker  81138
23:20:50   -I-   [cb    ]  Ticker  81139
23:20:51   -I-   [cb    ]  Ticker  81140
23:20:52   -I-   [sensor]   VBat=  1419 VBatF=  4.5 Temp=  1059 TempF=  62.3
23:20:52   -I-   [cb    ]  Ticker  81141
23:20:53   -I-   [cb    ]  Ticker  81142
23:20:54   -I-   [cb    ]  Ticker  81143
23:20:55   -I-   [cb    ]  Ticker  81144
23:20:56   -I-   [sensor]   VBat=  1393 VBatF=  4.4 Temp=  1060 TempF=  62.6
```

Analyzing the above the difference is the timestamp (from the RTC) and the ticker is:

timestamp	counter	diff
1087	1053	34
52836	51016	1820
52860	51039	1821

© Raja Srinivasan. All rights reserved.

timestamp	counter	diff
84037	81126	2911
84058	81146	2912
91024	87866	3158
91045	87886	3159

As the above table indicates the RTC (the column timestamp) appears to be ahead of the timer interrupts. In fact over a day, the difference is approaching 1 hour!

9.4. Development Insights

Hard realtime applications forces us to analyze the performance aspects very carefully. By necessity computations that can be tightly bound and predictable may be implemented in ISRs while more lengthy operations such as ADC or unpredictable operations like convergence in some computation are not appropriate for ISRs.

In this projectlet, the language runtime provided **Ada.Synchronous_Task_Control** is utilized as a synchronization mechanism. The ISR sets the event to true and the application can wait for this before performing one of the expensive operations. In a general purpose **RTOS** (Real Time Operating System) such as **Zephyr** the analog might be **event flag**s.

9.4.1. Inter task messaging

The interface provided to the Logging facility can be generalized as an inter task messaging facility. Ada tasks on a platform such as **linux** support such messaging; in the embedded profiles we are using, every task can have its own message queue in the above pattern.

© Raja Srinivasan. All rights reserved.

Listing 9.7 Wait for timer

File : sensorsob.adb

```
0048 |    loop
0049 |       Ada.Synchronous_Task_Control.Suspend_Until_True (cb.TimeToReadSensors);
0050 |       toolkit.adc.Read (VBat, Temp);
0051 |       VBatf := toolkit.adc.Raw_To_Volts_Vbat (VBat);
0052 |       TempF := toolkit.adc.Raw_To_Celsius (Temp);
0053 |       toolkit.logs.logger.Put_Line
0054 |         (" VBat= "
0055 |          & VBat'Image
0056 |          & " VBatF= "
0057 |          & VBatF'Image
0058 |          & " Temp= "
0059 |          & Temp'Image
0060 |          & " TempF= "
0061 |          & TempF'Image,
0062 |          source => myname);
0063 |    end loop;
```

Listing 9.8 Setup the PWM enabled timer

File : pwmfader.adb

```
0031 |    toolkit.timers.SetupPWM
0032 |      (modulator,
0033 |       toolkit.timers.pwm_1,
0034 |       stm32.timers.Channel_1,
0035 |       toolkit.timers.pwm_pin_1,
0036 |       stm32.Device.GPIO_AF_TIM3_2);
0037 |    loop
0038 |
0039 |       if dc /= cb.dutycycle then
0040 |          dc := cb.dutycycle;
0041 |          modulator.Set_Duty_Cycle (dc);
0042 |          toolkit.logs.logger.Put_Line
0043 |            ("Main " & cb.dutycycle'Image, source => myname);
0044 |       end if;
0045 |       delay 0.1;
0046 |    end loop;
```

© Raja Srinivasan. All rights reserved.

10. Data Logging

```
This chapter is based on the projectlet:

- 08_blog
```

10.1. Projectlet Goals

Thus far the projectlets depended on somewhat informal means of communications ie text based logging. As the applications grow in size and scope, we will find dedicated compute engines performing a small set of related tasks, several of them collaborating to achieve the overall system objective. An insulin delivery device for example may deploy a dedicated pump driver to deliver insulin while using another to interact with the user. In such applications text based communications is no longer adequate and the designs have to resort to binary formats. With text based logging, precious cycles have to be expended to convert raw data to a text form - usually requiring more space. A floating point variable representing say flow rate is typically 32 bits in raw form while converted to text could be much larger.

In this projectlet, we convert the USART based framework used earlier for logging into a binary data communication framework and use this to transmit the onboard sensor data digitized and transmitted to a host.

10.1.1. Protocol for binary data communications

Having taken the road to binary transmission, we now face a whole new set of design decisions. Particularly as the applications grow in size and scope, so do the development teams and the need to implement a disciplined approach is paramount.

A review of the blog post will be very beneficial.

In this projectlet, we adapt the nanoprotobuf library, being a lightweight implementation of the **protobuf** framework.

10.1.2. Data Integrity, Framing and Synchronization

The objective for this projectlet is to integrate the **nanoprotobuf** library and the USART framework to achieve data transmission. The receiver is assumed to be a high level platform such as **linux** and a simple utility is developed for illustrative purposes. Complications of framing, and a full featured interaction framework is left for a future projectlet.

10.2. Implementation

10.2.1. Payload Handling

As indicated above, **nanoprotobuf** is the tool for payload handling; the initial step being the definition of the payloads the controller wants to transmit and the ***command***s it expect to receive.

```
File : blog.proto

0002 | syntax = "proto2";
0003 |
0004 | message SensorsOnBoard {
0005 |     required int32 Vbat = 1 ;       // raw digitized value
0006 |     required int32 Temp = 2 ;
0007 |     required double Vbatf = 3 ;     // converted into voltage
0008 |     required double Tempf = 4 ;
0009 | }
0010 |
0011 | message AnalogInput {
0012 |     required int32 Raw = 5 ;       // raw digitized value
0013 |     required double Voltage = 6 ;  // converted into voltage
0014 | }
```

The above defines two different types of payloads **SensorsOnBoard** and a generic **AnalogInput**. The definitions focus on the payload only, leaving out framing, transport error detection and such concerns. This definition is of course language agnostic, supporting **C**, **C++**, **go** etc. **Ada** is not in this list however at this time!

10.2.1.1. Application Interface

This projectlet develops the **shim**s necessary to enable the embedded application to be able to convert raw data into a **protobuf** and vice versa.

10.2.1.2. Framing and Data Integrity

The protobuf buffer then is enveloped into a packet format before it can be handled by a suitable transport mechanism.

The message buffer definition below enables the above specification. In particular it contains **Suspension_Object**s that enable the application to synchronize the operation with the transport mechanism.

Senders of data wait on ***Transmission_Complete***.

Command handlers wait on ***Reception_Complete***.

In both cases, these are performed by helper **task**s and the application interfaces with them through messaging to these **task**s.

© Raja Srinivasan. All rights reserved.

Listing 10.1 API to pack values into the buffer

```
File : getset.ads

0019 |    function SetSensorsOnBoard(buffer: System.Address ;
0020 |                               buflen : Int ;
0021 |                               vbat : Int ;
0022 |                               temp : Int ;
0023 |                               vbatf : Interfaces.C.double ;
0024 |                               tempf : Interfaces.C.double )
0025 |                            return int
0026 |    with Import => True , Convention => C , External_name => "SetSensorsOnBoard";
0027 |
0028 |    function GetSensorsOnBoard(buffer: System.Address ;
0029 |                               buflen : Int ;
0030 |                               vbat : out Int ;
0031 |                               temp : out Int ;
0032 |                               vbatf : out Interfaces.C.double ;
0033 |                               tempf : out Interfaces.C.double )
0034 |                            return int
0035 |    with Import => True , Convention => C , External_name => "GetSensorsOnBoard";
```

10.2.2. Transport

The transport of text used in the logging support **toolkit.logs** and its backend **serial_port.nonblocking** (directly copied from **ADL**) is adapted for handling binary data packets. In particular the notion of line termination does not lend itself to this requirement. The following amendments are necessary:

Message Exchange Protocol

- Packets have a starting indicator **packet_begin**, followed by a message type and then the packet length. Message Type is not relied upon for any application in this version though it could be used for administrative reasons (eg. heartbeat). The payload follows the packet length.
- The Last octet of the packet contains a checksum of the payload.
- There is not attempt to handle occurrences of the **packet_begin** value within the payload.
- The receiver waits for a packet_begin, followed by the message type and packet lengths. Then octets are assembled into a packet for application usage

The weaknesses identified above are not insurmountable and are left as a future enhancement.

10.2.2.1. Hardware Interface

The approach follows the familiar pattern in embedded systems - leveraging the Interrupts generated by the USARTs. Transmissions proceed by starting with one character and every completion event triggers the transmission of the next character to be transmitted. Similarly octet arrivals generate an interrupt handled by retrieving the octet and then going back to wait for the next input.

© Raja Srinivasan. All rights reserved.

Listing 10.2 Specification for messages

File : toolkit-messages.ads

```
0010 |    -- Application message types should be chosen not to clash with
0011 |    -- standard message types - identified as #8x#
0012 |    subtype Message_Type is Storage_Element ;
0013 |    Msg_Sync : constant Message_Type      := 16#80# ;
0014 |    Msg_DateTime : constant Message_Type  := 16#81# ;
0015 |
0016 |    subtype Special_Bytes is Storage_Element ;
0017 |    packet_begin : constant Special_Bytes := 16#55# ;
0018 |    packet_escape : constant Special_Bytes := 16#aa# ; -- future
0019 |    packet_end : constant Special_Bytes := 16#00# ;    -- future
0020 |
0021 |    -- Payload is nanoprotobuf encoded payload.
0022 |    -- Checksum follows the Payload
0023 |
0024 |    -- +-------------+
0025 |    -- |     #55#    |     <- Implicitly transmitted
0026 |    -- |    MTYPE    |
0027 |    -- |   LENGTH    |     <- No overheads. Payload only
0028 |    -- |   PAYLOAD   |     <- Begin Checksum calc here
0029 |    -- |     ..      |
0030 |    -- |     ..      |     <- End of Checksum calc
0031 |    -- |   CHECKSUM  |
0032 |    -- +-------------+
0033 |
0034 |    TYPE_IDX : constant Storage_Offset := 1;
0035 |    LEN_IDX : constant Storage_Offset := 2;
0036 |    PAYLOAD_IDX : constant Storage_Offset := 3 ;
```

The package **serial-io.binary" implements the data send/receive as well as error detection.

The UART pins when monitored using a scope or a tool such as **pulseView** might exhibit:

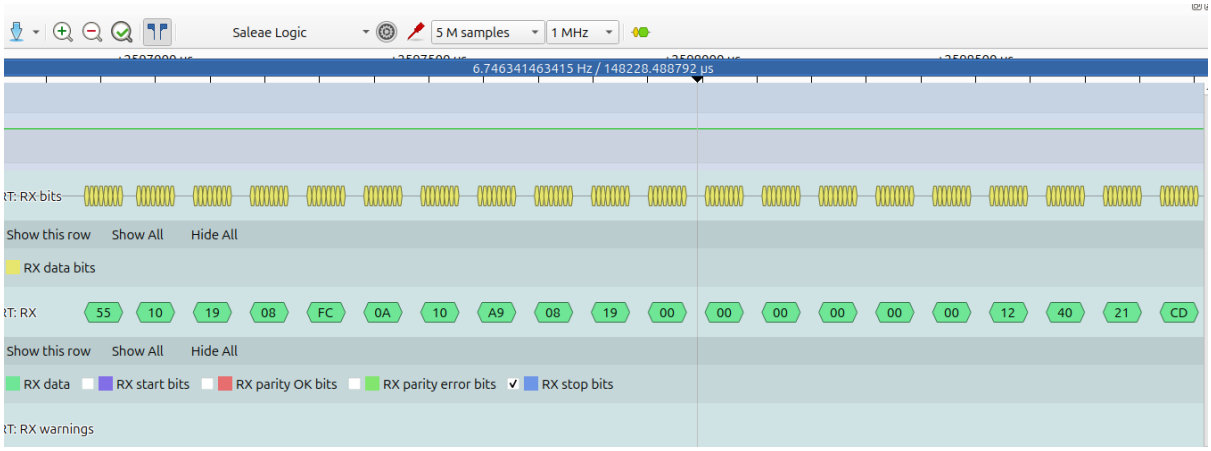


Figure 10.1.: PulseView capture

© Raja Srinivasan. All rights reserved.

Listing 10.3 Message Buffer Definition

File : toolkit-messages.ads

```
0121 |    type Buffer is tagged limited
0122 |    record
0123 |       Status : Status_Type := Buffer_Empty ;
0124 |       Storage : Storage_Array( 1..MAX_SIZE );
0125 |       Stored : Storage_Count := 0 ;
0126 |       Next_Out : Storage_Offset := 0 ;
0127 |       Next_In : Storage_Offset := 0 ;
0128 |       Reception_Complete    : Suspension_Object;
0129 |       Transmission_Complete : Suspension_Object;
0130 |       Error_Status          : Error_Conditions := No_Error_Detected;
0131 |    end record ;
0132 |    function CalcChecksum( This : in out Buffer ) return Storage_Element ;
0133 |    procedure CalcChecksum( This : in out Buffer );
0134 |    function VerifyChecksum( This : in out Buffer ) return Boolean ;
0135 |
```

Listing 10.4 Message Sender

File : toolkit-datalogs.adb

```
0065 |       loop
0066 |          Logger.Get (nxtmsg);
0067 |          toolkit.messages.Set (Outgoing, nxtmsg.all);
0068 |          Serial_Io.Binary.Send (DataLogCom, Outgoing'unchecked_access);
0069 |          Outgoing.Await_Transmission_Complete;
0070 |       end loop;
```

Similar handler for reception:

the whole cycle initiated like so:

with an analogous operation for packet reception.

10.3. Diagnostic Toolset

Host applications

The top level projectlet collection under **98_Utils** contain a series of utilities designed to exercise the facilities as they are being developed.

- For example 98_Utils/03_logger contains the projectlet ***capture*** that receives text log transmission on a host.
- 98_Utils/blog similarly contains the projectlet ***capture*** designed to recive binary packets in the above format and unpack the protobuf data and display.

© Raja Srinivasan. All rights reserved.

Listing 10.5 Message Receivers

```
File : toolkit-datalogs.adb

0083 |       loop
0084 |          Incoming.Clear;
0085 |          Serial_Io.Binary.Receive (DataLogCom, Incoming'access);
0086 |          Incoming.Await_Reception_Complete;
0087 |          mt := Incoming.Content (Integer (toolkit.messages.TYPE_IDX));
0088 |          mlen := Incoming.Content (Integer (toolkit.messages.LEN_IDX));
0089 |          toolkit.logs.logger.Put_Line
0090 |            ("Message Type : "
0091 |             & hex.Image (Unsigned_8 (mt))
0092 |             & " Length "
0093 |             & hex.Image (Unsigned_8 (mlen)),
0094 |             source => myname);
0095 |       end loop;
```

```
MType 10 Len 19 08fc0a10a40819000000000000124021cdccccccccc4c4f4056 Checksum : 56 >
Vbat  1404 Vbatf  4.50E+00 Temp  1060 Tempf  6.26E+01
MType 10 Len 19 08840b10a50819000000000000124021000000000804f4017 Checksum : 17 >
Vbat  1412 Vbatf  4.50E+00 Temp  1061 Tempf  6.30E+01
MType 10 Len 19 088a0b10a50819000000000000124021000000000804f401d Checksum : 1d >
Vbat  1418 Vbatf  4.50E+00 Temp  1061 Tempf  6.30E+01
MType 10 Len 19 08fe0a10a50819000000000000124021000000000804f4090 Checksum : 90 >
Vbat  1406 Vbatf  4.50E+00 Temp  1061 Tempf  6.30E+01
MType 10 Len 19 08f80a10a70819000000000000124021cdccccccccccc4f40d5 Checksum : d5 >
Vbat  1400 Vbatf  4.50E+00 Temp  1063 Tempf  6.36E+01
```

In most systems these packets will be written to external files for later analysis using an appropriate tool. **Julia** and **R** will be used in this effort for illustration.

10.4. Development Insights

While USARTs are used in this projectlet, systems are just as likely to use I2C, SPI or even Ethernet as the physical and datalink layer disciplines. **nanoprotobuf** is of course agnostic and thus will grow with us. USARTs pose a particular challenge of synchronization ie the different controllers may start and stop independently and there has to be enough hints in the transport protocol to be able to identify the start and end of a ***packet***. This projectlet again glossed over such complications for a simple solution leaving a more robust design for later.

For integrity this projectlet implements a checksum algorithm which again is not robust enough for safety critical applications. A **CRC** of 16 or even 32 bits might be needed if not a whole **hash** such as **md5**. Of course if **TCP/IP** is a choice - the data integrity concerns are correspondingly reduced.

© Raja Srinivasan. All rights reserved.

Listing 10.6 Character Transmission

```
File : serial_io-binary.adb

0030 |       procedure Handle_Transmission is
0031 |          Next_Out : Storage_Element;
0032 |          Empty    : Boolean;
0033 |       begin
0034 |          Outgoing_Msg.Get (Next_Out, Empty);
0035 |          if Empty then
0036 |             Device.Disable_Interrupts (Source => Transmission_Complete);
0037 |             Outgoing_Msg.Signal_Transmission_Complete;
0038 |             Outgoing_Msg := null;
0039 |          else
0040 |             Device.Transmit (UInt9 (Next_Out));
0041 |          end if;
0042 |       end Handle_Transmission;
```

Listing 10.7 Character Reception

```
File : serial_io-binary.adb

0046 |       procedure Handle_Reception is
0047 |          Received_Byte : constant Storage_Element :=
0048 |            Storage_Element (Device.Current_Input);
0049 |       begin
0050 |          Incoming_Msg.Append (Received_Byte);
0051 |          if Incoming_Msg.PacketComplete then
0052 |             loop
0053 |                --  wait for device to clear the status
0054 |                exit when not Device.Status (Read_Data_Register_Not_Empty);
0055 |             end loop;
0056 |             Device.Disable_Interrupts (Source => Received_Data_Not_Empty);
0057 |             Incoming_Msg.Signal_Reception_Complete;
0058 |             Incoming_Msg := null;
0059 |          end if;
0060 |       end Handle_Reception;
```

© Raja Srinivasan. All rights reserved.

Listing 10.8 Initiating packet transmission

```
File : serial_io-binary.adb

0084 |       procedure Start_Sending (Msg : not null access Buffer) is
0085 |       begin
0086 |          Outgoing_Msg := Msg;
0087 |          Device.Enable_Interrupts (Parity_Error);
0088 |          Device.Enable_Interrupts (Error);
0089 |          Device.Enable_Interrupts (Transmission_Complete);
0090 |          Handle_Transmission;
0091 |       end Start_Sending;
```

© Raja Srinivasan. All rights reserved.

11. Digital Communications - I2C

```
This chapter is based on the projectlet:

- 09_i2c

and supported by:

- 98_Utils/09_i2c
```

11.1. Projectlet Goals

Exploration of digital communication discipline using **USART**s exposed a significant weakness - the fact that there is no ***clock*** or ***timing*** as part of the physical layer. This could lead to inconsistencies between the cooperating devices. The invention of protocols such as **Inter-Integrated Circuit** and other similar derivatives (e.g. I2S, smbus) addressed this challenge amazingly well; these protocols have become an essential part of most embedded systems for integrating sensors, actuators and the like.

The architecture has further benefited from the qwiic ecosystem with standardized connectors and the large number of devices. In this projectlet, 2 such devices - a temperature sensor and a digital display are integrated with our microcontroller.

In addition the binary logging discipline developed earlier is applied to transmit the temperature feedback to a (presumably) supervisory level system. Examples of receiving these packages, unpacking using the **nanoprotobuf** library completes the system.

11.1.1. The roadmap

At a high level we will build the system to the following architecture.

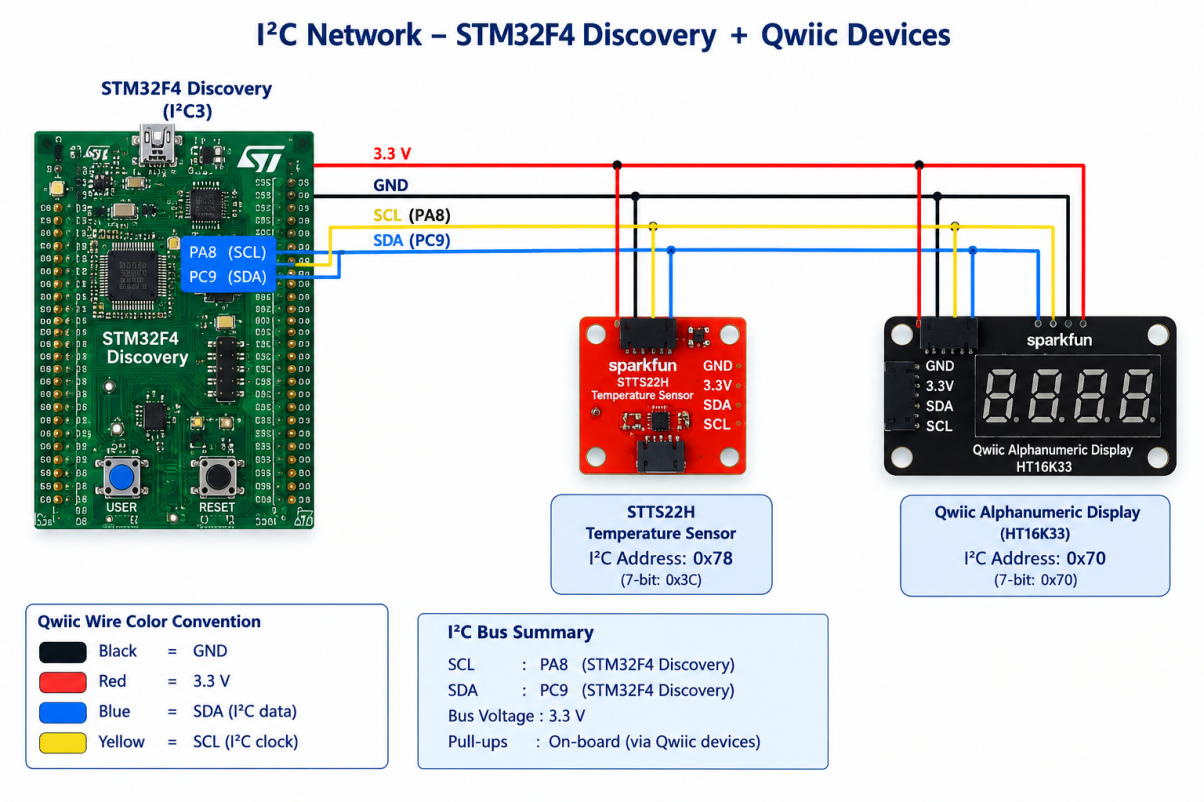


Figure 11.1.: Sensor Network Architecture

The temperature sensor will be used to measure the ambient temperature. The temperature converted into centigrade and timestamped will be transmitted over a binary serial interface.

The display is primarily an output device, used as a rotating display of alphanumeric characters or as a minute, second clock.

I2C being an extensible network, additional sensors or other compatible devices can be added with ease.

Further to form a complete system we will add a higher layer system, connecting to the micro-controller using the Serial protocol.

© Raja Srinivasan. All rights reserved.

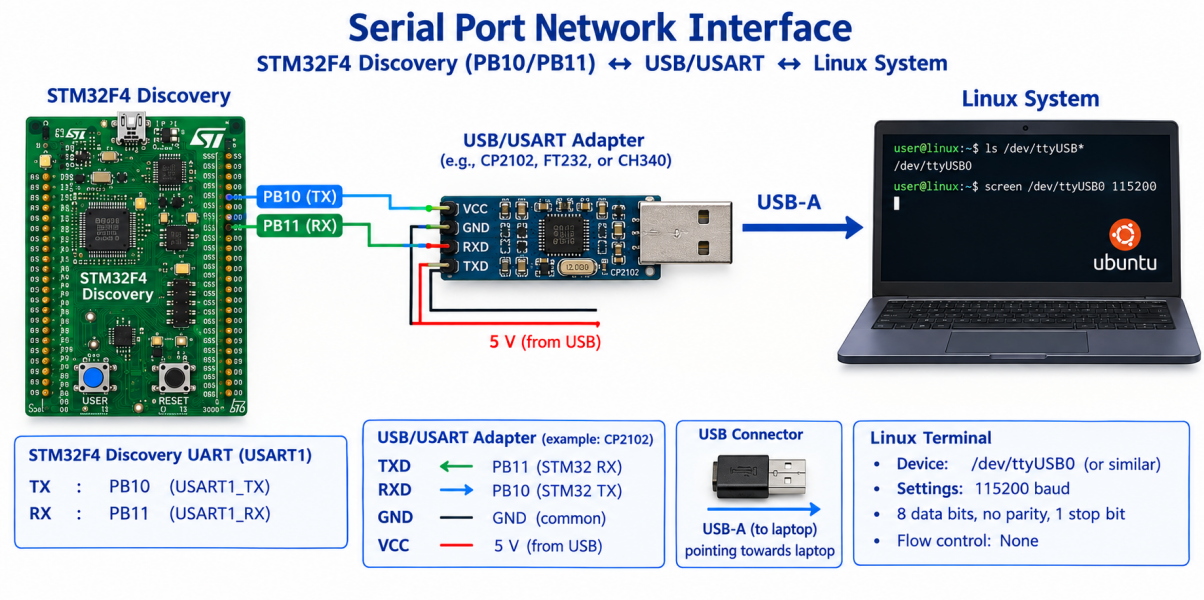


Figure 11.2.: Host system based on linux

The serial interface will be used to transmit the temperature readings packed into a **nanopro-tobuf** packet. The host will then receive the binary data, unpack and print out the temperature. It is of course feasible to have the host be on the I2C network; most real life devices prefer this approach or an ethernet network to integrate the supervisory systems.

11.2. Sensor Network

The microcontroller serves as the **master** of our I2C sensor network. The controller interacts with each of the **node**s of the network as the application demands. The base protocol dictates the line discipline and the steps of reading and writing.

The microcontroller utilizes one of the I2C devices on the board (I2C3) and sets itself up as the master of the network as shown above. The master then determines the data rates 100 Kbps as indicated above. Since the clock is part of the wire, the sensors dont get to choose the speed though not all sensors may be able to support the data rates depending on their function.

Once attached, reading and writing become pretty straightforward given the extensive support provided by the ADL through the packages **stm32.i2c**.

and for writing:

Each member usually has its own extensive specifications to be found in the data sheets.

11.2.1. Temperature Sensor stts22h

The stts22h temperature sensor, simple though it may seem, has an elaborate datasheet based on which the initializations, configurations are to be achieved before data acquisition. The raw data acquired may require further processing eg in this case conversion to centigrade before applications can utilize those values.

With the toolkit support above, the specific sensor is initialized:

© Raja Srinivasan. All rights reserved.

Listing 11.1 Attach and Initialize I2C3

```
File : toolkit-i2c.adb

0034 |    procedure Attach is
0035 |       cfg : stm32.i2c.I2C_Configuration;
0036 |    begin
0037 |       pragma Debug (stm32.GPIO.Set (stm32.board.Blue_LED));
0038 |       Enable_Clock (GPIO_A);
0039 |       Enable_Clock (GPIO_C);
0040 |       Enable_Clock (I2C_3);
0041 |       Init_I2c3_GPIO;
0042 |       cfg.Clock_Speed := 100_000;
0043 |       cfg.Mode := stm32.i2c.I2C_Mode;
0044 |       cfg.Addressing_Mode := stm32.i2c.Addressing_Mode_7bit;
0045 |       cfg.Own_Address := 0;
0046 |       stm32.i2c.Configure (stm32.Device.i2c_3, cfg);
0047 |       stm32.i2c.Set_State (stm32.Device.i2c_3, true);
0048 |    end Attach;
```

and the feedback read:

The 2 raw bytes provided by the sensor are to be converted into a temperature as the datasheets specify:

The application then may choose to convert the data into other units such as fahrenheit for logging as shown:

This pattern is pretty common in most sensors. In some extreme cases, applications may have to calibrate the sensors and incorporate those into the final calculations.

11.2.1.1. Temperature applications

The primary application **temp** reads the temperature from the device, converts it to fahrenheit, packs it into a nanoprotobuf packet and transmits over a binary serial port. A companion app **capture** runs on a host, receiving the packet, unpacking and printing out the temperature.

The following log from the utility shows the timestamp and the temperature readings:

```
MType 11 Len 14 3a0c081a10051801200028073016 45efa79242ff Checksum : ff >
 0: 7: 22:  Temp 73.33
MType 11 Len 14 3a0c081a10051801200028073017 4596c39242c3 Checksum : c3 >
 0: 7: 23:  Temp 73.38
MType 11 Len 14 3a0c081a10051801200028073018 456c6792423e Checksum : 3e >
 0: 7: 24:  Temp 73.20
MType 11 Len 14 3a0c081a10051801200028073019 45b89e9242c2 Checksum : c2 >
 0: 7: 25:  Temp 73.31
MType 11 Len 14 3a0c081a10051801200028073019 a454a8c924243 Checksum : 43 >
 0: 7: 26:  Temp 73.27
```

© Raja Srinivasan. All rights reserved.

Listing 11.2 Read a register (memory) from device

```
File : toolkit-i2c.adb

0052 |    procedure Read
0053 |      (stat   : out hal.i2c.I2C_Status;
0054 |       addr   : hal.i2c.I2C_Address;
0055 |       reg    : hal.UInt16;
0056 |       result : in out hal.i2c.i2c_data)
0057 |    is
0058 |       Status : hal.i2c.i2c_status;
0059 |    begin
0060 |       stm32.i2c.Mem_Read
0061 |         (This          => port,
0062 |          Addr          => addr,
0063 |          Mem_Addr      => Reg,
0064 |          Mem_Addr_Size => HAL.I2C.Memory_Size_8b,
0065 |          Data          => result,
0066 |          Status        => status);
0067 |       stat := Status;
0068 |    end Read;
```

11.2.2. Alphanumeric Display ht16k33

The alphanumeric display ht16k33 is an intricate device in itself incorporating distinct LEDs for each segment of the ***character*** requiring review of the schematic, data sheets and most importantly application examples. In the prototyping stage, the usual first attempt may well be with the Arduino system and often the associated library could serve as a model for our own efforts.

In fact due to the many variations on the display device, the Arduino library was the primary reference source. The driver was migrated to Ada and ADL using a variety of tools **ChatGPT**, **Claude** and manual conversion.

The interaction with the device follows the usual pattern. Once the network is ***Attach***ed any of the devices on the network can be supported. However the design of the display leads to a layer between the application and the hardware.

The above segment transmits a set of characters to the device which in turn proceeds by maintaining a temporary buffer translating each character into a pattern:

before it is transmitted to the device. The pattern is directly translated from the Arduino library referenced above.

11.2.2.1. Display Applications

There are 2 different applications in this projectlet:

- watch - every second updates the display with the minute and second
- disp - displays a rotating display of a long string

© Raja Srinivasan. All rights reserved.

Listing 11.3 Write to a register (memory) on the device device

File : toolkit-i2c.adb

```
0072 |    procedure Write
0073 |      (stat : out hal.i2c.I2C_Status;
0074 |       addr : hal.i2c.I2C_Address;
0075 |       reg  : hal.UInt16;
0076 |       data : in out hal.i2c.i2c_data) is
0077 |    begin
0078 |       stm32.i2c.Mem_Write
0079 |         (port,
0080 |          addr,
0081 |          Mem_Addr      => reg,
0082 |          Mem_Addr_Size => HAL.I2C.Memory_Size_8b,
0083 |          Data          => data,
0084 |          Status        => Stat);
0085 |    end Write;
```

Listing 11.4 Temperature sensor configuration

File : stts22h.adb

```
0072 |    procedure Setup (stat : out hal.I2C.I2c_Status) is
0073 |       CTRLVAL : constant :=
0074 |         SETGET_CTRL_FREERUN + SETGET_CTRL_IF_ADD_INC + SETGET_CTRL_AVG0
0075 |         + SETGET_CTRL_AVG1;
0076 |       pkt : HAL.I2C.I2C_Data (1 .. 1);
0077 |    begin
0078 |       pkt (1) := CTRLVAL;
0079 |       toolkit.i2c.Write (stat, addr, REG_CTRL, pkt);
0080 |    end Setup;
```

Listing 11.5 Temperature sensor configuration

File : stts22h.adb

```
0045 |    function Read (stat : out hal.i2c.I2C_Status) return Temperature_Type is
0046 |       rawresult : hal.i2c.i2c_data (1 .. 2);
0047 |       result    : Temperature_Type;
0048 |    begin
0049 |       toolkit.i2c.Read (stat, Addr, REG_TEMP_L_OUT, rawresult);
0050 |       result :=
0051 |         Temperature (Unsigned_8 (rawresult (1)), Unsigned_8 (rawresult (2)));
0052 |       return result;
0053 |    end Read;
```

© Raja Srinivasan. All rights reserved.

Listing 11.6 Temperature Compute from raw values

File : stts22h.adb

```
0020 |    function Temperature (L, H : Unsigned_8) return Temperature_Type is
0021 |       valfull : aliased Unsigned_16 :=
0022 |         Shift_Left (Unsigned_16 (H), 8) + Unsigned_16 (L);
0023 |       valint  : Short_Integer;
0024 |       for valint'Address use valfull'Address;
0025 |    begin
0026 |       return Temperature_Type (valint) / Temp_Mult;
0027 |    end Temperature;
```

Listing 11.7 Application usage of temperature

File : i2cnet.adb

```
0059 |       t := stts22h.Read (s);
0060 |       tf := toolkit.Convert (centigrade_type (t));
0061 |       toolkit.logs.logger.Put_Line
0062 |         ("STTS22H Status "
0063 |          & s'Image
0064 |          & " value "
0065 |          & t'Image
0066 |          & " or in deg F "
0067 |          & tf'image,
0068 |          source => myname);
```

© Raja Srinivasan. All rights reserved.

Listing 11.8 Unpack the nanoprotobuf buffer and extract the temperature

File : capture.adb

```
0042 |    procedure Packet_Body is
0043 |       Status : Int;
0044 |       yy , mm , dd , hh , min , ss  : Int ;
0045 |       value : Interfaces.C.double ;
0046 |    begin
0047 |       buffer(1) := mtype ;
0048 |       buffer(2) := packetlen ;
0049 |       pktlen := 2 ;
0050 |       for numbytes in 1..Integer(packetlen)
0051 |       loop
0052 |          Byte_Io.Read( ttybytesF , byte );
0053 |          pktlen := pktlen + 1 ;
0054 |          buffer(pktlen) := byte ;
0055 |          Put(hex.Image(Unsigned_8(byte))) ;
0056 |       end loop ;
0057 |       Put(" Checksum : "); Put(hex.Image(Unsigned_8(byte))) ; Put(" "); Put_Line(">")
0058 |       status := getset.GetTemperature (buffer(3)'Address , Int(pktlen-3) ,
0059 |                                        yy , mm , dd , hh , min , ss , value);
```

Listing 11.9 Write a string to the display

File : ht16k33_alpha.adb

```
0333 |    procedure Write_String (This : in out HT16K33_Display; S : String) is
0334 |    begin
0335 |       for D in 0 .. 3 loop
0336 |          This.RAM (D * 2) := 0;
0337 |          This.RAM (D * 2 + 1) := 0;
0338 |       end loop;
0339 |
0340 |       for I in S'Range loop
0341 |          exit when I - S'First > 3;
0342 |          Illuminate_Char (This, I - S'First, S (I));
0343 |       end loop;
0344 |    end Write_String;
```

© Raja Srinivasan. All rights reserved.

Listing 11.10 Character to illumination pattern

File : ht16k33_alpha.adb

```
0015 |    function Segment_Pattern (Ch : Character) return UInt16 is
0016 |    begin
0017 |       case Ch is
0018 |          when '0' =>
0019 |             return 2#00000000111111#; --, // '0'
0020 |
0021 |          when '1' =>
0022 |             return 2#00010000000110#; -- , // '1'
0023 |
0024 |          when '2' =>
0025 |             return 2#00000101011011#; -- , // '2'
0026 |
0027 |          when '3' =>
0028 |             return 2#00000101001111#; --, // '3'
0029 |
0030 |          when '4' =>
0031 |             return 2#00000101100110#; --, // '4'
0032 |
0033 |          when '5' =>
0034 |             return 2#00000101101101#; --, // '5'
0035 |
0036 |          when '6' =>
0037 |             return 2#00000101111101#; --, // '6'
0038 |
0039 |          when '7' =>
0040 |             return 2#01010000000001#; --, // '7'
0041 |
0042 |          when '8' =>
0043 |             return 2#00000101111111#; --, // '8'
0044 |
0045 |          when '9' =>
0046 |             return 2#00000101100111#; --, // '9'
0047 |
```

© Raja Srinivasan. All rights reserved.

A. Development Environment

In the interest of reducing the pedagogic friction arising out of the diverse computing platforms, this book comes with a pre-configured development environment. The development environment is available as **Docker** images, and can be used on any platform that supports Docker.

The application build image includes the GNAT Ada compiler, the Alire package manager, and all the tools and libraries needed to follow along with the examples in this book. The file Dockerfile.native found in the devops repository builds the container image and is available at container image

The later projects in this book generate data streams that can be captured and analyzed. The data analysis environment is available as a container image built with the Dockerfile found in the dataops repository and is available at container image **Julia** is the primary data analysis environment used in this book.

The documentation build image built with Dockerfile found in the docops repository builds a container image with **quarto** and is available at container image

The book is produced with these containers in a CI pipeline.

A.1. Application code repository

https://gitlab.com/ada23/embedded/stm32f4disco/projects.git serves as the application code repository. Divided into a series of projects of increasing functionality and complexity. It uses the Ada Drivers Library as a submodule. It is designed to be collaborative with other components but can be used independently as well.

Each project eg. cli contains the source code, build, and flash support as well as documentation. The documentation is designed to be integrated with the book but can be built and viewed independently. Typicall the documentation includes extracts from the source code for illustrative purposes.

A.1.1. Book source

The directory 99_doc contains the documentation structure. **quarto** is the documentation engine used.

A.1.2. Target microcontrollers

The conceptual framework utilitizes the **STM32f4 Discovery** board as the target microcontroller. Versions of this book and the accompanying code are planned for the ESP32 and the Raspberry Pi Pico.

A.1.3. Usage

The containers have ssh servers running and can be accessed using ssh. The containers are configured to listen on different ports on the host machine:

Container	ssh port
devops:latest	2222
docops:latest	2223
dataops:latest	2224

A.1.4. Building the applications

The container **devops:latest** is used for building the applications. The shared volume is mounted at /ws in the container. A script buildall.py may be provided by the application to build the example applications.

```
cd /ws/projects
python3 buildall.py
```

A.1.5. Code deployment or flashing the target

If the host is Linux, the binaries may also be downloaded to the target using the tools in the container such as **openocd** and **st-flash**. Other platforms such as Windows and MacOs may not support direct download from the container and will rely on the host versions of these tools. Typically each project has a script **flash.sh** that can be used to flash the target using openocd.

A.1.6. Code example extracts

In addition to the binaries of the application, the container also can generate code extracts that are used in the documentation. These code extracts are generated using the codemd tool and are made available to the documentation container through the shared volume. For instance:

```
codemd -D./.codemd src/*.ad*
```

will analyze all the ada files, extract the designated segments and deposit them in the .codemd directory.

The script **pushdoc.py** pushes all the documentation including code extracts in **.codemd** in the source code repository to the documentation environment prior to building the book.

A.1.7. Building the documentation

The container **docops:latest** is used for building the documentation. The book source is made available to the container at /ws/doc and the output is typically available at /ws/doc/public. :

```
cd /ws/doc/src
quarto render
```

© Raja Srinivasan. All rights reserved.

A.1.8. Data Analysis

Most of the applications in this book generate data streams that can be captured and analyzed using the documentation container. **Julia** is the primary data analysis environment used in this book and is available in the container **dataops:latest**.

© Raja Srinivasan. All rights reserved.